

# DLBC Information Unit

## Product and UX handoff

A current product and interface audit, followed by the full project handoff.

**Read Section 18 first. It contains the current takeover audit. Earlier sections include a historical UX handoff whose observations may describe an earlier version of the app.**

**Working branch:** codex/product-ux-audit

**Review date:** 30 September 2026

## What the audit found

- 1. Human approval needs a clear boundary.** The newer processing path can mark a report complete without linking a proofread revision, while the project requirements describe separate human-reviewed stages.
- 2. An outage can look like an empty workspace.** With the backend offline, the dashboard and history show empty-session messages alongside a connection error.
- 3. The dashboard claims a microphone is ready too early.** That label appears even while the setup screen still requires microphone permission.
- 4. A settings save can fail without clear feedback.** The automation switch changes onscreen before the server confirms it and does not show a failed-save state.
- 5. Keyboard focus can escape the mobile menu.** When it opens, pressing Tab can move focus to a control hidden behind it.

Section 18 includes priorities, evidence, and proposed improvement slices. It separates browser observations from source findings.

**Recommended first step: improve the dashboard and session setup, then resolve the report approval boundary before redesigning later review stages.**

## Full handoff

The following pages preserve the full handoff: screen inventory, workflows, design system, responsive notes, and current takeover audit.

# DLBC Information Unit — Current App UX Handoff

|                                                                                                                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Takeover baseline, 30 September 2026: Read Section 18 before using the earlier audit as current implementation truth. Some earlier routing, dashboard, and workflow descriptions have been superseded. The new section distinguishes browser observations from source findings and proposals. |
| Authoritative Technical & UX State Audit                                                                                                                                                                                                                                                      |
| Target Audience: UI/UX Product Designers, ChatGPT Redesign Prompts, Frontend Engineers                                                                                                                                                                                                        |
| Application: Deeper Life Bible Church (DLBC) Information Unit Reporting System                                                                                                                                                                                                                |
| Audit Date: September 2026                                                                                                                                                                                                                                                                    |
| Status: Current Working Implementation (Phases 0–9 Complete)                                                                                                                                                                                                                                  |
| Mode: Pure Observation & Handoff (Zero Code Refactoring Applied)                                                                                                                                                                                                                              |

## TABLE OF CONTENTS

- App Overview
- Complete Screen Inventory
- User Flow Map
- Navigation Structure & Sitemap
- Current Design System
- Component Inventory
- Data and Domain Model
- Roles and Permissions
- Forms Inventory
- Current Responsive Behaviour
- Visual Evidence & Screenshots Index
- Current UX Problems Observed
- Accessibility Review
- Current Content and Terminology
- Implementation Constraints for Polishing
- Source File Map
- Security & Environment Variables Summary

## 1. APP OVERVIEW

### 1.1 What the DLBC Information Unit App Is Designed to Do

The **DLBC Information Unit App** is a specialized, desktop-first production web application built specifically for the Information Unit of **Deeper Life Bible Church (DLBC)**.

During church programmes (such as Sunday Morning Worship Services, Monday Bible Studies, Thursday Revival & Evangelism Training Services, National Retreats, and Global Crusades), sermons and messages are ministered live. The Information Unit is tasked with capturing the message, transcribing it with high accuracy, producing structured journalistic and spiritual reports, editing and proofreading the content to adhere strictly to church doctrine and publication standards, and generating official documentation ready for immediate distribution (via church bulletins, email, and WhatsApp groups).

The application combines:

- **Lossless Master Audio Capture:** Live browser audio recording from sound mixers, USB audio interfaces, or microphones, as well as uploaded audio/video files and YouTube streams.
- **Real-Time Speech-to-Text Transcription:** Powered primarily by Azure Cognitive Services Speech (specifically tuned to English - Nigeria `en-NG`), with local CPU-based `faster-whisper` and Google Cloud Speech-to-Text fallbacks.
- **Structured Human Verification:** An exception-based review tool where low-confidence words, church terminology, dates, numbers, and biblical references are flagged for quick operator review against bounded audio replay before the transcript is locked.
- **Dual Independent AI Reporting:** Generates two distinct perspectives on the sermon using Google Gemini:
- **Reporter A:** Focuses on core sermon structure, main points, thematic outline, and key scriptures.
- **Reporter B:** Focuses on illustrative details, anecdotes, supplementary scriptures, specific warnings, and omissions.
- **AI-Assisted Editorial Synthesis with Human Oversight:** Compiles Reporter A and Reporter B into a unified Information Unit report against the authoritative Verified Transcript, allowing human editors to inspect sources in a slide-out drawer, edit text, and track revisions.
- **Conservative AI Proofreader:** Performs conservative grammatical, syntax, capitalization, and punctuation checks without altering doctrinal meaning, presenting a structured changes list for human acceptance.
- **Document Generation & Archival Export:** Compiles approved reports into standardized, beautifully formatted Microsoft Word documents (`.docx`) for dissemination.

## 1.2 Who the Users Are & Planned User Roles

In the **current V1 implementation**, the application is operated as a **single-operator administration tool**. There is currently **no multi-user authentication, login screen, or role-based access control (RBAC)** implemented in code. A single operator (the Information Unit Lead or media desk operator) controls the machine from start to finish.

However, as defined in `PROJECT\_SCOPE.md` and `PROJECT\_PLAN.md`, the workflow is architected around five distinct human stakeholder groups who will eventually interact with the system:

- **System Operator / Project Lead (Current Active Role):** Sets up the laptop in the media booth, configures audio input, monitors recording, manages transcript verification, triggers AI generation, and downloads final files.
- **Information Unit Reporters:** Historically sat in the auditorium manually writing independent sermon reports. In the digital workflow, reporters review raw transcripts, run independent reporting drafts, and add human reporting notes.
- **Information Unit Editors:** Senior writers who reconcile conflicting reporter drafts into a single cohesive report. They review AI synthesis, verify quotes against the audio/transcript, and enforce church editorial standards.
- **Information Unit Proofreaders:** Language specialists who review final texts for spelling, punctuation, capitalization of divine pronouns, scripture citations, and typographical errors.
- **Information Unit Leadership / Church Media Team:** Oversee quality baselines, configure official programmes, update editorial standards, and approve final documents for public release.

## 1.3 The Primary Problem the App Solves

Before this application, the Information Unit workflow was **100% manual, labor-intensive, duplicative, and error-prone**:

- **Duplicated Effort:** Multiple reporters independently sat through the same 2-hour service frantically typing identical sermon notes.
- **Manual Compilation Bottleneck:** A single lead editor had to manually read 3–5 different handwritten or typed summaries, reconciling omissions, discrepancies, and conflicting scriptures under tight Sunday publication deadlines.
- **Lack of Audio Traceability:** Once an editor typed a report, there was no systematic way to verify whether a particular sentence was an exact quote or an editor's paraphrase without manually searching through hours of raw sermon tape.
- **Fatigue & Inconsistent Quality:** Late-night Sunday proofreading was subject to human fatigue, resulting in inconsistent capitalization of biblical terms, misattributed scripture verses, and omitted message points.
- **The Solution:** "**Capture once. Verify what matters. Process consistently. Deliver faster.**" By capturing lossless master audio, automating transcription with Nigerian English language models, restricting human verification to low-confidence flags, and using dual AI drafts with human sign-off, report compilation time is cut by over 70% while

preserving absolute doctrinal integrity.

## 1.4 Form Factor & Deployment Targets

- **Desktop-First Web Application:** Optimized for 1440px and 1920px laptop and desktop screens typically used in church media galleries and sound booths.
- **Responsive Down to Mobile:** Features responsive breakpoints for 1024px tablet, 768px small tablet, and 360px–390px mobile screens (including off-canvas hamburger navigation drawer and stacked cards).
- **Local & Cloud Hybrid Runtime:**
  - Local Development / Offline Media Booth: Runs on a Windows laptop with Python FastAPI backend, local SQLite database (`storage/app.db`), and local filesystem storage.
  - Cloud Production Ready: Dockerized container (`Dockerfile`) designed for Azure Container Apps (South Africa North region) mounted to Azure Files (`DATA\_ROOT=/data`) with Azure Speech and Gemini API cloud connections.

## 1.5 Current Technology Stack

- **Frontend:**
  - Framework: React 19 (`react: ^19.2.8`, `react-dom: ^19.2.8`)
  - Build Tool: Vite (`vite: ^8.2.0`, `@vitejs/plugin-react: ^6.0.4`)
  - Linter: Oxlint (`oxlint: ^1.75.0`)
  - State Management: React Native Hooks (`useState`, `useEffect`, `useCallback`, `useRef`)
  - Routing: **State-driven view switching** (`currentView` in `App.jsx` and `activeView` in `SessionDetailView.jsx`). **No React Router / Wouter is installed.**
  - Styling System: Vanilla CSS via a massive monolithic stylesheet (`src/App.css`, 10,429 lines, 193 KB). No Tailwind CSS, CSS Modules, or UI component libraries (such as MUI, Chakra, or Shadcn/ui) are currently installed.
- **Backend:**
  - Framework: FastAPI 0.141.1 (Python 3.12+)
  - Server: Uvicorn 0.52.3 with WebSockets support (`websockets: 16.1.1`)
  - Database: SQLite via `aiosqlite: 0.22.1` and `SQLAlchemy: 2.0.52` (with full DDL support for production Azure SQL / MSSQL via `pymssql` and `aiodbc`)
  - Document Generation: `python-docx: 1.2.0` (generates formatted Word `.docx` documents)
  - YouTube Ingestion: `yt-dlp: 2026.8.19` with `bgutil-ytdlp-pot-provider`
- **Speech-to-Text & AI Services:**
  - Primary Live Transcription: Azure Cognitive Services Speech SDK (`azure-cognitiveservices-speech: 1.51.1`, region `southafricanorth`, language `en-NG`)
  - Local Offline Transcription: `faster-whisper: 1.2.1` with CTranslate2 INT8 model quantization
  - Cloud File Transcription: Google Cloud Speech-to-Text (`google-cloud-speech: 2.40.0`)
  - AI Reporting, Editing & Proofreading: Google Gemini API via `google-genai: 2.18.1` (using `gemini-2.5-flash` with versioned system prompt templates)
- **Storage:**
  - Structured local file repository rooted at `storage/` (`storage/audio/`, `storage/uploads/`, `storage/transcripts/`, `storage/verified\_transcripts/`, `storage/documents/`).

## 1.6 Key Technical Constraints Affecting UI/UX

- **Audio Immutability Guarantee:** Master audio captured from church mixers or microphones is an immutable legal/spiritual record. The UI must never offer destructive overwriting or lossy client-side compression of source recordings.
- **Immutable Machine Transcripts:** Machine transcripts generated by Azure or Whisper are never directly mutated in place. Human review produces a distinct `verified\_text` artifact.

- **Strict Workflow Gate Dependencies:** The application follows an authoritative 8-stage lifecycle. Downstream stages (Reporting, Editing, Proofreading, Final Report) are physically locked until prerequisite stages have been approved or confirmed by the human operator.
- **Single-Operator Browser State:** Because routing is state-based rather than URL-based, refreshing the browser (F5) currently resets the user back to the Dashboard view, requiring navigation back to the active session.
- **Browser Media Permissions:** Capturing church mixer audio or YouTube tab audio relies on browser `navigator.mediaDevices.getUserMedia` and `getDisplayMedia`, requiring clear UI prompts when permissions are blocked.
- **AI Provider Rate & Cost Limits:** API calls to Gemini and Azure Speech consume budget; UI requires confirmation dialogs before re-running multi-stage AI jobs.

## 2. COMPLETE SCREEN INVENTORY

Every user-facing screen, modal, drawer, and workspace in the application was directly inspected in the codebase. Each entry details the real implementation, user actions, component composition, states, responsive behavior, and implementation quirks.

### Screen 1: Home Dashboard

- **Screen Name:** Home Dashboard
- **Route / Path:** Root view (`currentView === 'dashboard'`)
- **Relevant Source File(s):** `frontend/src/components/dashboard/DashboardView.jsx`, `frontend/src/App.jsx`
- **Who Can Access It:** All operators (default landing screen).
- **Purpose of the Screen:** Central launchpad for starting live church service captures, uploading existing message audio/video, launching YouTube sermon ingestion, and highlighting active sessions that require urgent operator attention.
- **What Is Currently Displayed:**
- **Top Hero Action Cards (3 equal cards):**
  - Start Live Session (Deep brand navy card `#0F2947`, mic icon, "Mic / USB Direct Audio" status dot).
  - Upload Recording (White card, folder icon, file drag-and-drop support, hidden file input).
  - YouTube Session (White card, red play icon, "Paste YouTube URL" status dot).
- **"Needs Your Attention" Section:** Displayed conditionally when active sessions are interrupted or have unverified flagged segments. Displays alert badge `🔔`, session title, date, minister name, pending flag count, and a direct action button (`Review Sections >` or `Continue to Editing >`).
- **Recent Sessions Table:** Summary table listing the last 5 sessions with columns: `SESSION IDENTIFIER`, `LOG DATE`, `DURATION`, and `STATUS` (color-coded pills: `● COMPLETED`, `● PROOFREADING COMPLETE`, `● EDITING COMPLETE`, `● REPORTS READY`, `● VERIFIED`, `● NEEDS VERIFICATION`, or `● INTERRUPTED`).
- **Primary User Action:** Click `Start Live Session` to launch the pre-recording configuration flow.
- **Secondary Actions:**
  - Drag and drop or browse an audio/video file onto the `Upload Recording` card.
  - Click `YouTube Session` to open the YouTube URL ingestion workspace.
  - Click `Review Sections >` on an attention card to jump directly into transcript verification.
  - Click any row in the `Recent Sessions` table to open that session's central workspace.
  - Click `View All Sessions →` to navigate to the full session history directory.
- **Navigation Available:** Left sidebar (`Dashboard`, `Sessions`, `Settings`), mobile hamburger menu, hero card triggers, and table row clicks.
- **Important Components Used:** `DashboardView`, `AppShell`.
- **Modals, Sheets, Dropdowns, Popovers:** Native file browser dialog triggered by the hidden `<input type="file">` on the Upload card.

- **Empty States:** When no sessions exist in `storage/app.db`, the Recent Sessions table displays an empty row: "No recorded sessions found. Click Start Live Session above to begin your first service recording."
- **Loading States:** No visible spinner on initial dashboard render; session data loads asynchronously via `useSessions` hook.
- **Error States:** Global `ErrorBanner` rendered at the top of the viewport if database query fails.
- **Success/Confirmation States:** None on dashboard itself.
- **Responsive / Mobile Behaviour:**
  - At 1440px desktop: 3-column hero grid, spacious table layout.
  - At 768px tablet: Hero cards collapse to 2 columns.
  - At 390px mobile: Hero cards stack vertically into a single column. The Recent Sessions table experiences horizontal overflow, introducing an awkward horizontal scrollbar (`table-wrapper`). The top header bell icon can become partially clipped.
- **Anything Unusual About the Screen:** The entire `Upload Recording` card acts as a clickable dropzone, but there is no visual change (such as border color highlight or overlay) during native drag-over events.

## Screen 2: New Live Session Setup

- **Screen Name:** New Live Session Setup
- **Route / Path:** `currentView === 'new\_live'`
- **Relevant Source File(s):** `frontend/src/components/sessions/NewLiveSessionView.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Pre-recording configuration screen where the operator assigns church metadata (Programme, Section, Minister, Sermon Title) and tests audio levels before initiating recording.
- **What Is Currently Displayed:**
  - **Header:** "New Live Session", subtitle, and "← Back to Dashboard" button.
  - **Left Column (Session Metadata Card):**
    - Programme / Event dropdown (dynamically populated from configured programmes, e.g. Sunday Worship Service, Monday Bible Study).
    - Session / Section dropdown (filtered child sections of selected programme, e.g. Morning Session, Evening Revival Service).
    - Minister / Speaker text input (with placeholder `e.g. Pastor W.F. Kumuyi`).
    - Message Title text input (marked `Optional`).
    - Generated Session Title preview input with `⌵ Reset to auto` button when manually overridden.
  - **Right Column (Audio Input & Start Action):**
    - Audio Input Source dropdown listing all browser-detected devices (identifies USB audio interfaces vs built-in laptop microphones).
    - Device list `⌵ Refresh` button.
    - Live Audio Input Level Visualizer: Animated decibel meter bar (`-40`, `-20`, `0 dB`) color-shifting from green (`meter--good`) to yellow/red (`meter--hot`).
    - Informational notice: "Verify settings before starting. Master lossless audio will be captured."
    - Large Primary CTA: `⌵ START RECORDING`.
- **Primary User Action:** Click `START RECORDING` to launch the live capture stream.
- **Secondary Actions:**
  - Select different connected audio inputs (e.g. switch from built-in mic to USB mixer feed).
  - Click `⌵ Refresh` to redetect newly plugged-in USB audio interfaces.
  - Grant audio microphone permissions if initially blocked.
  - Manually customize the generated session title.
  - Click `← Back to Dashboard` to abort setup.

- **Navigation Available:** Top bar back action, left sidebar navigation.
- **Important Components Used:** `NewLiveSessionView`, `useAudioCapture`.
- **Modals, Sheets, Dropdowns:** Native select dropdowns for Programme, Session Section, and Audio Input Device.
- **Empty States:** If no programmes are configured in settings, dropdown shows "No programmes configured in Settings" (disabled). If microphone permission is missing, displays an inline permission prompt card with `Grant Audio Permission` button.
- **Loading States:** Displays "Loading configured programmes..." while fetching from `/api/programmes`.
- **Error States:** Validation error banner appears if session title is empty or if audio capture fails (`Please specify or select a Session Title`).
- **Success/Confirmation States:** Clicking Start immediately starts Web Audio capture, initializes the database session, and transitions to the Live Recording view.
- **Responsive / Mobile Behaviour:**
  - Desktop: Clean 2-column layout (Metadata left, Audio right).
  - Mobile (390px): Columns stack vertically. The Start Recording button is pushed below the fold, requiring scrolling down to reach the primary action.
- **Anything Unusual About the Screen:** The audio input meter runs an active microphone test stream as soon as the screen opens so operators immediately see if the sound mixer is feeding audio, without having to click a separate "Test" button.

## Screen 3: Live Session Recording Monitor

- **Screen Name:** Live Session Recording Monitor (Full-Screen Mode)
- **Route / Path:** `currentView === 'live\_recording'` or when `isRecording && !isRecorderMinimized`
- **Relevant Source File(s):** `frontend/src/components/recording/LiveRecordingView.jsx`, `frontend/src/hooks/useAudioCapture.js`
- **Who Can Access It:** Operator currently executing a live message recording.
- **Purpose of the Screen:** Dedicated, mission-critical cockpit for monitoring sermon capture, observing real-time streaming speech-to-text transcripts, tracking audio signal integrity, and flagging uncertain sermon moments live.
- **What Is Currently Displayed:**
  - **Top Recording Control Bar:**
    - Pulsating Red Recording Badge: `● RECORDING ACTIVE` (or `● YOUTUBE TAB AUDIO`).
    - Elapsed Time Digital Clock (`HH:MM:SS` updating every second).
    - Multi-bar animated Audio Waveform visualizer reflecting live input volume.
    - Signal Status Badge (`STRONG` or `IDLE`).
    - Compact Programme/Sermon metadata breadcrumb.
    - Minimize button.
    - Prominent STOP SESSION danger button.
  - **Real-Time Dominant Transcript Canvas:**
    - Live streaming text area with automatic smooth scroll-to-bottom.
    - Timestamped speech segments with start offsets (`[MM:SS]`).
    - Real-time interim typing ticker with blinking cursor `|` showing immediate unfinalized speech.
    - Flagged moments highlighted with warning badge `⚠`.
    - Live manual flag toggle button `⏏` on each segment.
  - **Secondary Operational Status Footer:**
    - Storage Health Pill: `STORAGE: OPTIMAL`
    - AI Engine Status: `AI TRANSCRIPTION: LISTENING / RECOGNIZING`
    - Engine Model Tag: `Azure Speech (en-NG) • Lossless PCM Archive`
- **Primary User Action:** Monitor sermon audio and click `STOP SESSION` when the minister concludes the preaching.

- **Secondary Actions:**

- Click `⌵ Minimize` to collapse the recorder into the floating widget and access other parts of the app during service.
- Click `□` on any segment to manually flag a name, scripture, or unclear statement for later verification.
- **Navigation Available:** Minimized button collapses to floating widget; all sidebar links are active (clicking any sidebar item automatically minimizes the live recorder so recording never stops).
- **Important Components Used:** `LiveRecordingView`, `useAudioCapture`, Web Audio `AudioWorkletNode` (pcm-recorder-processor.js).
- **Modals, Sheets, Dropdowns:** None; all controls are inline to prevent accidental dialog blocking during live church recording.
- **Empty States:** When listening before speech begins: "Listening for live audio input... Live transcript will stream here." with waiting spinner `⌵`.
- **Loading States:** Interim speech displays real-time animated cursor `|`.
- **Error States:** Global banner if WebSocket disconnects or audio worklet crashes.
- **Success/Confirmation States:** Stopping the session triggers a clean transition to the `SessionCompletionView` modal.
- **Responsive / Mobile Behaviour:**
  - Desktop: Spacious layout with transcript canvas filling remaining viewport height.
  - Mobile (390px): Top control bar becomes very crowded; timer, waveform, minimize, and stop buttons compress into tight rows, increasing the risk of accidental taps near the Stop button.
- **Anything Unusual About the Screen:** This view captures lossless raw PCM audio in memory and streams chunks via WebSocket while simultaneously displaying Azure Speech live recognitions. If the operator navigates away, recording **does not stop**—it automatically switches to the floating controller.

## Screen 4: Floating Persistent Recording Controller

- **Screen Name:** Floating Persistent Recording Controller
- **Route / Path:** Persistent fixed overlay (isRecording && isRecorderMinimized)
- **Relevant Source File(s):** `frontend/src/components/recording/FloatingRecordingController.jsx`
- **Who Can Access It:** Operator while an active recording session is running in the background.
- **Purpose of the Screen:** Compact picture-in-picture style floating widget anchored to the bottom-right corner, allowing the operator to navigate session histories, check settings, or view previous reports without interrupting an ongoing service recording.
- **What Is Currently Displayed:**
  - Fixed floating card in the lower-right viewport.
  - Pulsating red dot with `LIVE` tag and live elapsed timer (HH:MM:SS).
  - Mini 7-bar audio waveform visualizer.
  - Maximized button (⌵ Open Recorder).
  - Emergency Stop button (□ Stop).
  - Session title line with church icon `☙`.
  - Single-line real-time ticker showing the latest words recognized by Azure Speech.
  - Footer status indicators: Audio signal, Azure STT status, and total segment counter.
- **Primary User Action:** Click `Open Recorder` to return to the full-screen live recording canvas.
- **Secondary Actions:** Click `Stop` to terminate and finalize the session directly from the floating widget.
- **Navigation Available:** Does not block underlying page navigation; stays fixed above all views.
- **Important Components Used:** `FloatingRecordingController`.
- **Modals, Sheets, Dropdowns:** None.
- **Empty States:** Shows "Listening for sermon speech..." in the ticker if minister is silent.
- **Loading States:** Real-time text ticker streams with cursor `|`.
- **Error States:** Waveform goes flat if microphone input drops.

- **Success/Confirmation States:** Clicking Stop opens `SessionCompletionView`.
- **Responsive / Mobile Behaviour:**
  - Desktop: Positioned in bottom-right with 380px width, unobtrusive.
  - Mobile: Takes up significant bottom viewport space (almost 35% of visible screen height), potentially obscuring bottom table rows or action buttons on the underlying view.
- **Anything Unusual About the Screen:** Seamless state synchronization: zero frames of audio or transcript text are lost when switching between full-screen and minimized floating mode.

## Screen 5: Session Completion Summary Modal

- **Screen Name:** Session Completion Summary Modal
- **Route / Path:** `showCompletionModal === true` (rendered in `App.jsx` immediately following stop recording)
- **Relevant Source File(s):** `frontend/src/components/sessions/SessionCompletionView.jsx`
- **Who Can Access It:** Operator who just stopped a recording session.
- **Purpose of the Screen:** Formal handoff screen confirming that lossless master audio and indexed raw transcripts were safely written to disk, and guiding the operator immediately to the next required action (Verification or Reporting).
- **What Is Currently Displayed:**
  - **Top Hero Banner:** Deep navy banner with green checkmark circle `✔`, "Session Captured Successfully", and subtitle "Recording and transcript are safely preserved on secure storage."
  - **Metadata Grid:**
    - Service Name (e.g. Sunday Morning Worship Service).
    - Duration (e.g. 01:42:15).
    - Date (e.g. Sunday, Oct 24, 2023).
  - **System Storage Status Box:**
    - Audio Saved: `✔ Primary + Backup Lossless WAV`
    - Raw Transcript: `✔ Indexed & Synced to Database`
    - System Badge: `SYSTEM READY`
  - **Verification Callout Card:**
    - If flags exist: Shows amber icon `⚠`, "Verification Required", and explanation: "The AI transcript flagged X sections requiring human review for theological accuracy or spelling." with primary CTA button `Begin Verification (X) →`.
    - If no flags: Shows green icon, "No Verification Required", with primary CTA `Go to Reporting →`.
- **Bottom Secondary Actions:**
  - `✔ Finish for Now` (returns safely to Dashboard).
  - `✔ View Session Details` (opens the central Session Workspace).
- **Primary User Action:** Click `Begin Verification (X) →` (or `Go to Reporting →`).
- **Secondary Actions:** Click `Finish for Now` or `View Session Details`.
- **Navigation Available:** Trapped modal view until one of the three exit buttons is clicked.
- **Important Components Used:** `SessionCompletionView`.
- **Modals, Sheets, Dropdowns:** Full-screen overlay modal with frosted backdrop.
- **Empty States:** Not applicable (always renders with session data).
- **Loading States:** None.
- **Error States:** None directly on modal.
- **Success/Confirmation States:** Banner prominently confirms file persistence.
- **Responsive / Mobile Behaviour:**
  - Desktop: Centered 640px modal card.
  - Mobile (390px): Expands to full screen width. The three action buttons stack, requiring vertical scrolling to inspect storage details.

- **Anything Unusual About the Screen:** If there are 0 flags and the operator clicks `Go to Reporting`, the app automatically confirms the raw machine transcript as the verified transcript behind the scenes (`confirmRawAsVerified`), saving a manual step.

## Screen 6: Sessions History Directory

- **Screen Name:** Sessions History Directory
- **Route / Path:** `currentView === 'sessions'` (when no `activeSession` is loaded)
- **Relevant Source File(s):** `frontend/src/components/sessions/SessionHistoryList.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Comprehensive catalog and filtering interface for all church service sessions stored in the SQLite database.
- **What Is Currently Displayed:**
- **Top Header:** "Sessions History", subtitle displaying count of active sessions (`Active sessions: X`), `⌂ Refresh` button, and `+ New Session` primary CTA button.
- **Filter & Search Panel:**
- Search text field (searches sermon title, session ID, preacher name, and metadata JSON).
- Status Filter dropdown (`All Statuses`, `Needs Verification`, `Verified`, `In Editing`, `Completed`, `Interrupted`).
- Date Range dropdown (`All Time`, `Last 7 Days`, `Last 30 Days`).
- `Clear Filters` link button (appears when filters are active).
- **3-Column Card Grid (Stitch Design):**

Each session is represented as a card featuring a color-coded top accent border:

- Red border: Interrupted sessions (`⏏ Interrupted`).
- Blue border: Live recording in progress (`● Live`).
- Amber border: Needs human verification (`Needs Verification`).
- Green border: Verified and ready for reporting (`✅ Verified`).
- Navy border: Final Report completed (`✅ Complete`).
- Card metadata: Date, formatted time, duration, sermon title, and minister/speaker name with icon `👤`.
- Card action button: Contextual button (`Continue Verification`, `Go to Reporting`, `Continue to Editing`, `Download Document`, `Review Log`).
- Delete button `⏏` with native browser confirmation prompt.
- **Primary User Action:** Click on a session card or its action button to open that session in the Session Workspace hub.
- **Secondary Actions:**
- Type in the search box to find a sermon by preacher or topic.
- Filter by lifecycle status.
- Click `+ New Session` to configure a new live recording.
- Delete an outdated or test session record using `⏏`.
- Click `⌂ Refresh` to reload from SQLite database.
- **Navigation Available:** Left sidebar navigation, top bar, card clicks.
- **Important Components Used:** `SessionHistoryList`.
- **Modals, Sheets, Dropdowns:** Status and date select dropdowns; native `window.confirm` for deletion.
- **Empty States:**
- If database has 0 sessions: "No sessions have been recorded yet. Click '+ New Session' to start recording your first service."
- If search/filter matches 0 records: "No sessions match your search or filter criteria." with `Clear Filters` option.
- **Loading States:** Spinner and disabled state on Refresh button when loading.
- **Error States:** Global error banner if backend request fails.

- **Success/Confirmation States:** Session deletion removes card immediately and refetches list.
- **Responsive / Mobile Behaviour:**
  - Desktop (1440px): 3-column card grid.
  - Tablet (768px): 2-column card grid.
  - Mobile (390px): Single-column card grid. Search bar and dropdowns wrap onto separate lines.
- **Anything Unusual About the Screen:** Deletion uses native `window.confirm()` rather than an accessible in-app modal. Clicking anywhere on the card opens the workspace, while the delete button stops propagation.

## Screen 7: Central Session Workspace Hub

- **Screen Name:** Central Session Workspace Hub
- **Route / Path:** `currentView === 'sessions'` (when `activeSession`` is loaded, `activeView === 'overview'`)
- **Relevant Source File(s):** `frontend/src/components/sessions/SessionDetailView.jsx``, `frontend/src/components/common/LifecycleStepper.jsx``
- **Who Can Access It:** All operators working on a specific sermon.
- **Purpose of the Screen:** The central mission-control dashboard for an individual church service message. It unifies the 8-stage lifecycle stepper, provides direct artifact status tiles, guides the operator with a prominent "Action Required" hero banner, and provides navigation to all downstream sub-workspaces.
- **What Is Currently Displayed:**
- **Session Header Bar:**
  - Live Status Badge: `• ACTIVE VERIFICATION``, `• VERIFIED``, `• REPORTS READY``, `• EDITING COMPLETE``, `• PROOFREADING COMPLETE``, or `• COMPLETED``.
  - Inline Title Editor: Editable sermon title with `⌵ Edit Details`` button that toggles an inline `<input>`` with `Save Title`` and `Cancel`` buttons.
  - Metadata line: Date ``⌵``, Programme ``⌵``, Section ``⌵``, Minister ``⌵``, and Message Topic ``⌵``.
- **Authoritative 8-Stage Lifecycle Stepper:**

Horizontal track connecting 8 numbered nodes: `1. Recording` → 2. Raw Transcript` → 3. Verification` → 4. Verified Transcript` → 5. Reporting` → 6. Editing` → 7. Proofreading` → 8. Final Report``. Completed steps show `⌵``, active step has pulsating dot, locked steps show `⌵``.

- **"Action Required" Hero Banner:**

Dynamically reflects the exact immediate task needed:

- Needs Verification: Amber hero banner with alert icon `⌵⌵``, "X sections require human verification", and button `Begin Verification (X pending blocks) →``.
- Verified: Green hero banner, "Verified Transcript Approved", button `Go to Reporting →``.
- Reports Ready: Blue hero banner, "Reporting Drafts Ready for Compilation", button `Open Editing Workspace →``.
- Editing Complete: Purple hero banner, "Edited Report Ready for Proofreading", button `Open Proofreading Workspace →``.
- Completed: Gold hero banner with trophy `⌵``, "Final Message Report Ready", button `Download Final Document (.docx) →``.
- **6 Artifact Tiles Grid (3 Columns x 2 Rows):**
  - Audio Recording (Master WAV source, duration, `⌵ Play Recording`` action).
  - Raw Transcript (Synced segment count, `View Raw Transcript`` action).
  - Verified Transcript (Approved factual record badge, `View Verified Transcript`` or `Begin Verification`` action; locked if unverified).
  - Reporter Drafts (Dual draft status, `Go to Reporting`` action; locked until verified).
  - Edited Report (Theological review status, `Compile Draft`` / `Open Editing`` action; locked until reports are ready).
  - Final Report (Archival & Word document status, `Download .docx`` / `Finalize Report`` action; locked until proofreading is complete).

- **Operational Incident Banner (Conditional):** If the session suffered a power or network interruption during capture, displays an informational notice with recovery notes confirming that the master recording was preserved.
- **Primary User Action:** Click the large primary action button inside the "Action Required" hero banner to advance the session to its next stage.
- **Secondary Actions:**
  - Click on any unlocked artifact tile to inspect raw audio, transcripts, or drafts.
  - Click on any unlocked stage node in the Lifecycle Stepper to jump directly to that stage.
  - Click `□□ Edit Details` to rename the sermon or adjust the message title.
  - Click `← Back` in the top header to return to the Sessions History list.
- **Navigation Available:** Stepper nodes, tile buttons, hero button, top header back button, sidebar links.
- **Important Components Used:** `SessionDetailView`, `LifecycleStepper`.
- **Modals, Sheets, Dropdowns:** Inline title editing input box; no popover menus.
- **Empty States:** If opened without a valid session, renders null.
- **Loading States:** Inline "Saving..." text while updating title in database.
- **Error States:** Global ErrorBanner if API calls fail.
- **Success/Confirmation States:** Title changes save immediately to SQLite and update the UI.
- **Responsive / Mobile Behaviour:**
  - Desktop (1440px): 8-step stepper spans full width; 6 artifact tiles sit in a 3x2 grid.
  - Mobile (390px): The 8-step lifecycle stepper overflows horizontally with a scroll track. The 6 artifact tiles collapse into a single vertical column. The Hero action banner takes up the entire top fold.
- **Anything Unusual About the Screen:** This hub acts as the parent router for all child stages. Clicking any stage replaces the workspace hub content with that stage's dedicated subview while updating the topbar title and back action.

## Screen 8: Raw Transcript Viewer

- **Screen Name:** Raw Transcript Viewer (Standalone Mode)
- **Route / Path:** `currentView === 'sessions'`, `activeView === 'raw\_transcript'` (or via `transcribe` view)
- **Relevant Source File(s):** `frontend/src/components/transcription/RawTranscriptViewer.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Dedicated interface for inspecting the unmutated machine-generated transcript, listening to the synchronized master audio recording, and reviewing machine confidence scores before verification.
- **What Is Currently Displayed:**
  - **Header Bar:** Sermon title, `RAW TRANSCRIPT` pill tag, creation date, total duration, and a `□ Copy Full Transcript` action button with temporary `□ Copied!` state.
  - **Prominent Verification Alert Banner:** Displayed if unverified flags exist: amber box with alert icon `□□`, "Verification Needed: The AI detected X sections requiring human verification..." with direct CTA button `Begin Verification →`.
  - **Master Audio Playback Scrubber:**
    - Play/Pause toggle button (`□` / `□`).
    - Current playback time / total duration counter (`MM:SS / MM:SS`).
    - Interactive waveform timeline scrubber allowing direct scrubbing to any second.
  - **View Mode Switcher:** Toggle buttons for `Timestamped Segments` vs `Continuous Text`.
- **Content Area:**
  - Segments Mode: List of timestamped speech blocks. Clicking any timestamp jumps audio to that exact moment and starts playback. Low-confidence segments display confidence badge (e.g. `CONF: 62%`) with amber highlighting.
  - Continuous Text Mode: Formatted paragraph-style reading canvas.
- **Archival Immutability Policy Footer:** Informational footer stating: "This raw transcript is preserved as an immutable legal record. All corrections are saved to a separate Verified Transcript."
- **Primary User Action:** Click `Begin Verification →` (if flagged) or click timestamps to listen to audio segments.

- **Secondary Actions:**

- Toggle between Timestamped Segments and Continuous Text view.
- Play, pause, or seek master sermon audio.
- Click `□ Copy Full Transcript` to copy the complete raw text to clipboard.
- Click `← Back to Session Workspace` to return to the hub.
- **Navigation Available:** Back button to workspace hub; top bar back navigation.
- **Important Components Used:** `RawTranscriptViewer`, HTML5 `` element with custom CSS scrubber.
- **Modals, Sheets, Dropdowns:** None.
- **Empty States:** If no transcript text or segments exist, shows placeholder notice.
- **Loading States:** Audio scrubber buffers native audio stream; no blocking spinner.
- **Error States:** Audio playback errors fail silently or log to browser console.
- **Success/Confirmation States:** `Copy Full Transcript` shows green checkmark `□ Copied!` for 2.5 seconds.
- **Responsive / Mobile Behaviour:**
  - Desktop: Full-width transcript reading canvas with sticky audio scrubber bar.
  - Mobile: Audio scrubber controls compress; timestamp buttons stack above segment text.
- **Anything Unusual About the Screen:** Audio streaming is backed by the FastAPI media streaming endpoint (`/api/transcription/media/{id}`) supporting range requests.

## Screen 9: Verification Workspace

- **Screen Name:** Verification Workspace
- **Route / Path:** `currentView === 'sessions'`, `activeView === 'verification'`
- **Relevant Source File(s):** `frontend/src/components/verification/VerificationWorkflow.jsx`, `frontend/src/components/verification/VerificationItemRow.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Exception-based verification interface where the human operator reviews only the low-confidence or manually flagged speech moments, plays bounded audio loops of the preacher's voice, corrects or confirms text, and finalizes the authoritative factual transcript.
- **What Is Currently Displayed:**
  - **Header Bar:** `VERIFICATION MODE` blue badge, Session Identifier, Sermon Title, and pending count badge (`X Pending Sections`).
- **2-Column Layout:**
  - **Left Column (Item Queue & Bulk Actions):**
    - Progress Meter: Visual bar showing resolved vs total items (e.g. `3 / 15 Resolved (20%)`).
    - Filter Tabs: `[ Pending (X) | Resolved (Y) | All (Z) ]`.
    - Bulk Confirmation Action: `□ Confirm All Remaining` button (triggers modal).
    - Flagged Items List: Scrollable list of cards. Each card displays timestamp `[MM:SS]`, flag reasons (e.g. `low\_confidence`, `unrecognized\_terminology`, `biblical\_reference`), confidence percentage, snippet, and status (`PENDING`, `CORRECTED`, `CONFIRMED`).
  - **Right Column (Active Segment Review & Audio Scrubber):**
    - **Master Audio Scrubber:** Play/pause, master seek bar, speed multiplier toggle (`1.0x` → `1.25x` → `1.5x` → `0.75x`), and current time.
    - **Active Segment Card:**
      - Segment header with timestamp and confidence rating.
      - `□ Replay Segment Audio` button (executes **bounded playback**: plays strictly between start and end timestamps and automatically pauses).
      - Raw Machine Output box (read-only reference with flagged words highlighted).
      - Verified Text Input Field: Large `` pre-populated with original text, allowing direct operator editing.</li>
</ul>
</li>
</ul>
</li>
</ul>
</li>
</ul>
</div>
<div data-bbox="101 960 319 974" data-label="Page-Footer">Takeover audit dated 30 September 2026</div>
<div data-bbox="878 960 903 973" data-label="Page-Footer">13</div>

- Action Buttons:
- `□ Original Was Correct` (confirms raw transcript without changes).
- `Save Correction & Next →` (saves correction to SQLite and auto-advances to the next pending item).
- **Bottom Finalization Banner:** When all items are resolved (`0 Pending`), an alert banner appears: "All sections verified! Ready to compile the authoritative Verified Transcript." with primary CTA button `□ Finalise Verification`.
- **Primary User Action:** Listen to bounded segment audio, type correction (or accept original), and click `Save Correction & Next →`.
- **Secondary Actions:**
  - Click `Confirm All Remaining` to bulk-accept all pending segments.
  - Switch playback speed to 1.25x or 1.5x for faster audio verification.
  - Click any card in the left queue to jump directly to that item.
  - Click `Finalise Verification` to generate the immutable `verified\_text` artifact.
- **Navigation Available:** `← Back to Session Workspace` button in subview header.
- **Important Components Used:** `VerificationWorkflow`, `VerificationItemRow`, HTML5 Audio with bounded segment playback timers.
- **Modals, Sheets, Dropdowns:**
  - Bulk Confirmation Modal: Frosted overlay asking "Confirm all remaining X items without further review?".
  - Native browser confirmation dialog on clicking `Finalise Verification`.
- **Empty States:** If a session has 0 flags upon opening, shows a clean state with option to confirm raw text as verified immediately.
- **Loading States:** Inline loading spinners when saving items or finalising.
- **Error States:** Inline alert if saving an item fails.
- **Success/Confirmation States:** Resolved items turn green in the queue with a checkmark `□`.
- **Responsive / Mobile Behaviour:**
  - Desktop: Side-by-side columns (Queue left, Review right).
  - Mobile (390px): Columns stack. The queue pushes the active review editor down the page, making it tedious to select items and review audio without excessive scrolling.
- **Anything Unusual About the Screen:** Bounded audio replay: clicking `Replay Segment Audio` automatically pauses playback when the segment end time is reached, preventing the operator from having to manually stop the tape.

## Screen 10: Verified Transcript Approved Viewer

- **Screen Name:** Verified Transcript Approved Viewer
- **Route / Path:** `currentView === 'sessions'`, `activeView === 'verified\_transcript'`
- **Relevant Source File(s):** `frontend/src/components/sessions/SessionDetailView.jsx` (lines 258–324)
- **Who Can Access It:** All operators (unlocked only after verification is completed).
- **Purpose of the Screen:** Displays the officially approved, continuous factual transcript that forms the immutable truth for all downstream reporting and editing stages.
- **What Is Currently Displayed:**
  - **Header:** Sermon title, subtitle ("Verified Transcript • Created [Date] • Verified by Operator Admin"), and action buttons:
  - `Finish for Now` (returns to workspace).
  - `Continue to Reporting →` (advances to AI reporting).
  - **Verification Complete Banner:** Green banner with checkmark circle `□`, "Verification Complete: This transcript is now the approved factual source for Information Unit Reporting and Editing."
  - **Toolbar:** `□ View Raw Transcript` link and `□ Copy Full Transcript` button.
  - **Reading Canvas:** Clean, formatted continuous reading canvas displaying the full verified sermon text.
  - **Primary User Action:** Click `Continue to Reporting →` to advance to the next lifecycle stage.
  - **Secondary Actions:**

- Click `□ Copy Full Transcript` to copy the verified text to clipboard.
- Click `□ View Raw Transcript` to compare with the unedited machine transcription.
- Click `Finish for Now` to return to the workspace overview.
- **Navigation Available:** Top header actions, back button to workspace overview.
- **Important Components Used:** Embedded subview inside `SessionDetailView`.
- **Modals, Sheets, Dropdowns:** None; copies text directly to clipboard with native `alert()` feedback.
- **Empty States:** Renders placeholder if `verified\_text` is empty.
- **Loading States:** None.
- **Error States:** None.
- **Success/Confirmation States:** Clicking copy triggers a browser alert `□ Copied verified transcript text to clipboard!`.
- **Responsive / Mobile Behaviour:** Text wraps comfortably on mobile; top action buttons stack vertically.
- **Anything Unusual About the Screen:** Uses native browser `alert()` instead of an in-app toast notification when copying text to the clipboard.

## Screen 11: Reporting Workspace

- **Screen Name:** Reporting Workspace (Dual Independent Reporters)
- **Route / Path:** `currentView === 'sessions', `activeView === 'reporting`
- **Relevant Source File(s):** `frontend/src/components/reporting/ReportingView.jsx`,  
`frontend/src/components/reporting/ReportCard.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Orchestrates and displays dual independent AI reporting drafts (Reporter A and Reporter B) generated by Google Gemini from the authoritative Verified Transcript.
- **What Is Currently Displayed:**
- **Top Hero Banner:**
- Back button: `← Back to Session`.
- Active Standard Badge button: `Standard: Reporting Standard v1 [Manage]` (opens standards modal).
- Sermon title and horizontal inline lifecycle trail.
- **Status Ribbon / Ready Banner:**
- Before Generation: Status ribbon showing Source (`□ Verified Transcript`), Active Standard (`Reporting Standard v1`), and primary CTA `□ Generate Reports`.
- After Generation: Floating green card with checkmark circle `□`, "Both Independent Report Drafts Are Ready!", `□ Regenerate` button, and primary CTA `Continue to Editing →`.
- **Missing API Key Notice (Conditional):** If `GEMINI\_API\_KEY` is missing in backend `.env`, renders an informational card explaining how to configure it.
- **Dual Reporter Cards Grid (2 Equal Columns):**
- **Reporter A Card (Main Message & Structure):**
- Navy badge `A`, title: "Main Message & Structure", subtitle: "Focuses on central themes, logical outline, key statements, and core scriptures".
- Status pill (`Generating...`, `Ready`, or `Failed`).
- Generated Report Title and Report Text.
- Structured metadata sections: `Key Points / Themes` (bulleted list), `Core Scriptures Cited` (tag pills), and `Warnings & Exhortations`.
- Individual `□ Regenerate Reporter A` action button.
- **Reporter B Card (Detail & Omission Watch):**
- Purple badge `B`, title: "Detail & Omission Watch", subtitle: "Focuses on specific illustrations, secondary scriptures, ministry dates, and contextual nuances".
- Status pill.

- Generated Report Title and Report Text.
- Structured metadata sections: `Specific Details & Anecdotes`, `Supplementary Scriptures`, and `Omission Safeguards`.
- Individual `⌂ Regenerate Reporter B` action button.
- **Primary User Action:** Click `⌂ Generate Reports` (or `Continue to Editing →` when ready).
- **Secondary Actions:**
  - Click `Manage` on the standard badge to view or tune prompt instructions.
  - Individually regenerate Reporter A or Reporter B if one draft is unsatisfactory.
  - Click `← Back to Session` to return to the workspace hub.
- **Navigation Available:** Top banner back action, Continue to Editing button, standards modal trigger.
- **Important Components Used:** `ReportingView`, `ReportCard`, `ReportingStandardsModal`.
- **Modals, Sheets, Dropdowns:** `ReportingStandardsModal`.
- **Empty States:** When reports have not been generated, ReportCards display empty states: "No report generated yet. Click 'Generate Reports' above to begin AI reporting."
- **Loading States:** Pulsating status badges and animated loading text (`⌂ Generating Reports...`, `Generating Reporter A...`).
- **Error States:** Red `error-banner` at top of view if Gemini API fails or rate limit is reached.
- **Success/Confirmation States:** Ready banner floats at the top with green checkmark when both reporters finish.
- **Responsive / Mobile Behaviour:**
  - Desktop: 2-column side-by-side comparison grid.
  - Mobile (390px): Stacks Reporter A above Reporter B, making visual cross-comparison difficult without scrolling back and forth.
- **Anything Unusual About the Screen:** Dual AI generation: Instead of a single AI summary, two separate prompts run concurrently to mirror the traditional church practice of having multiple human reporters take independent notes.

## Screen 12: Reporting Standards Modal

- **Screen Name:** Reporting Standards Modal
- **Route / Path:** Modal dialog inside `ReportingView` or `SettingsView`
- **Relevant Source File(s):** `frontend/src/components/reporting/ReportingStandardsModal.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Allows operators to inspect, customize, and version the active system instructions, reporting guidelines, church terminology glossary, and few-shot examples that guide Reporter A and Reporter B.
- **What Is Currently Displayed:**
  - **Modal Header:** Title "Reporting Standards Management", active standard version label (e.g. `v1`), and close button `⌂`.
  - **Protected Backend Rules Notice:** Informational callout confirming that immutable guardrails (non-fabrication, verified transcript authority) cannot be overwritten.
- **Tabs:**
  - `Active Guidelines`: Displays editable text areas for:
    - General Guidelines.
    - Reporter A Instructions (Thematic outline rules).
    - Reporter B Instructions (Detail and anecdote capture rules).
    - Church Terminology & Glossary.
    - Approved Examples.
    - Change Notes input.
  - `Version History`: List of historical standards versions with timestamps, version labels, author notes, and `Activate` buttons to rollback prompts.
- **Modal Footer:** `Cancel` button and `Save & Activate Standard` primary button.
- **Primary User Action:** Edit guidelines or terminology and click `Save & Activate Standard`.

- **Secondary Actions:** Switch to Version History tab to restore an earlier prompt version.
- **Navigation Available:** Close button `□`, backdrop click, or Cancel.
- **Important Components Used:** `ReportingStandardsModal`.
- **Modals, Sheets, Dropdowns:** Modal overlay with scrollable content body.
- **Empty States:** Version history displays empty state if only v1 exists.
- **Loading States:** Disables save button with `Saving...` indicator during POST request.
- **Error States:** Inline error message if required fields are blank.
- **Success/Confirmation States:** Closes modal and updates active version badge in parent view.
- **Responsive / Mobile Behaviour:** On mobile, modal fills 95% of viewport width; textareas become tight.
- **Anything Unusual About the Screen:** Database-backed prompt engineering: Changes to prompts create immutable, versioned database records in `reporting\_standards`, ensuring complete reproducibility of reports generated months apart.

## Screen 13: Editing Workspace

- **Screen Name:** Editing Workspace (Editorial Synthesis & Review)
- **Route / Path:** `currentView === 'sessions'`, `activeView === 'editing'`
- **Relevant Source File(s):** `frontend/src/components/editing/EditingView.jsx`,  
`frontend/src/components/editing/SourceReferenceDrawer.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** The primary editorial workshop where the AI synthesizes Reporter A and Reporter B into a cohesive publication report, and the human editor reviews, refines, formats, and approves the text while cross-referencing source materials in a flyout drawer.
- **What Is Currently Displayed:**
- **Top Navigation Bar:**
- `← Back to Reporting` action.
- Sermon Title, Minister Name, and Service Date.
- `□□ EDITING PHASE` badge.
- Stepper trail: `1 [Done] → 2 [Done] → 3 [Done] → 4 [Done] → 5 [Done] → 6 [Active] → 7 [Future] → 8 [Future]`.
- **Editorial Toolbar:**
- Standard Version tag button (opens Editor Standards modal).
- `□ Source Drawer` toggle button (opens flyout drawer).
- `□ History (X)` button (opens revisions drawer).
- Live Word Count and Character Count counters (`X words • Y characters`).
- Unsaved changes indicator (`● Unsaved edits`).
- Action buttons:
- `□ Save Edits` (saves a new revision).
- `□ Copy Report`.
- `□ Export Word (.docx)` (downloads immediate preview).
- `□ Complete Editing` (primary action advancing to Proofreading).
- **Editable Article Canvas:**
- Sermon Title input (`h1` styled form control).
- Large full-viewport ``<textarea>`` containing the complete edited sermon report, structured with markdown headings, scripture quotes, and point outlines.
- **Primary User Action:** Review AI-compiled report, make manual editorial refinements, and click `□ Complete Editing`.
- **Secondary Actions:**
- Click `□ Source Drawer` to slide open the side panel and inspect the raw Verified Transcript, Reporter A, or Reporter B side-by-side.

- Click `□ Save Edits` to snapshot manual changes as a new revision.
- Click `□ History` to view or restore earlier revisions.
- Click `□ Export Word (.docx)` to download a working draft.
- Click `□ Re-synthesize from Sources` (with confirmation modal) to rerun AI compilation.
- **Navigation Available:** Back to reporting, complete editing forward action, source drawer toggle, revision drawer toggle.
- **Important Components Used:** `EditingView`, `SourceReferenceDrawer`, `EditorStandardsModal`.
- **Modals, Sheets, Dropdowns:**
  - `SourceReferenceDrawer`: Right-hand off-canvas slide-out drawer.
  - `EditorStandardsModal`: Modal for editing synthesis guidelines.
  - Re-synthesize Confirmation Dialog.
- **Empty States:** If no edited draft exists yet: shows empty card with `□ Generate Edited Report Draft` CTA.
- **Loading States:** Animated banner during AI compilation: "AI Editor synthesizing Reporter A & B against Verified Transcript...".
- **Error States:** Red error banner if compilation or save fails.
- **Success/Confirmation States:** Green banner on save (`□ Manual changes saved successfully as a new revision`).
- **Responsive / Mobile Behaviour:**
  - Desktop: Ample room for full editorial canvas; source drawer overlays comfortably.
  - Mobile (390px): The extensive toolbar wraps onto 4–5 crowded lines. The source drawer takes over the entire mobile screen when opened.
- **Anything Unusual About the Screen:** Multi-revision history: Every manual save and AI generation creates an immutable revision record in `edited\_reports`, allowing editors to revert accidental deletions at any time.

## Screen 14: Editor Standards Modal

- **Screen Name:** Editor Standards Modal
- **Route / Path:** Modal dialog inside `EditingView` or `SettingsView`
- **Relevant Source File(s):** `frontend/src/components/editing/EditorStandardsModal.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Allows editors to inspect and customize the compilation rules, tone of voice, terminology, and historical report examples used by Gemini when synthesizing multiple reporter drafts into a single report.
- **What Is Currently Displayed:**
  - Modal Header: "Editor Standards Management", active version label (`v1`), close button `□`.
  - Protected Rules Box: Reminder that verified transcript supremacy and non-fabrication are immutable backend constraints.
  - Tabs:
    - `Active Guidelines`: Editable textareas for General Guidelines, Compilation Guidance, Terminology, Approved Historical Examples, and Revision Notes.
    - `Version History`: Historical list with `Activate` buttons to rollback prompts.
  - Footer: `Cancel` and `Save & Activate Standard` buttons.
- **Primary User Action:** Adjust compilation instructions and click `Save & Activate Standard`.
- **Secondary Actions:** Rollback to an earlier editor standard version.
- **Navigation Available:** Close button `□`, backdrop click, Cancel button.
- **Important Components Used:** `EditorStandardsModal`.
- **Modals, Sheets, Dropdowns:** Modal dialog with backdrop.
- **Empty States:** Version history empty state.
- **Loading States:** `Saving...` button state.
- **Error States:** Validation error if guidelines are blank.

- **Success/Confirmation States:** Success notification upon saving new version.
- **Responsive / Mobile Behaviour:** Scrollable vertical modal on mobile devices.
- **Anything Unusual About the Screen:** Stores few-shot approved examples of past DLBC publications to guide the AI's spiritual tone and journalistic formatting.

## Screen 15: Source Reference Flyout Drawer

- **Screen Name:** Source Reference Flyout Drawer
- **Route / Path:** Off-canvas slide-out drawer inside `EditingView` (`activeView === 'editing'`)
- **Relevant Source File(s):** `frontend/src/components/editing/SourceReferenceDrawer.jsx`
- **Who Can Access It:** Editor reviewing the synthesized draft.
- **Purpose of the Screen:** Side-by-side verification drawer that slides in from the right edge, allowing the human editor to inspect the exact wording of the original Verified Transcript, Reporter A's structural draft, or Reporter B's details draft without leaving their active editing canvas.
- **What Is Currently Displayed:**
- **Drawer Header:** Title "Source Materials Reference", description, and close button `✕`.
- **Source Switcher Tabs:**
- `Verified Transcript` (tag: `Authoritative Source`).
- `Reporter A` (tag: `Structure & Points`).
- `Reporter B` (tag: `Details & Omissions`).
- **Search & Filter Input:** Quick text search box for finding specific phrases within the active source.
- **Content Body:** Scrollable reading pane rendering the complete selected source text.
- **Primary User Action:** Read source material and cross-reference against the edited draft on the left.
- **Secondary Actions:** Switch between tabs (Verified Transcript vs Reporter A vs Reporter B); close drawer.
- **Navigation Available:** Close button `✕` or clicking outside the drawer.
- **Important Components Used:** `SourceReferenceDrawer`.
- **Modals, Sheets, Dropdowns:** Fixed off-canvas right drawer (480px width on desktop).
- **Empty States:** Displays "No content available for this source" if draft failed.
- **Loading States:** None.
- **Error States:** None.
- **Success/Confirmation States:** None.
- **Responsive / Mobile Behaviour:** On mobile, drawer expands to 100% width, completely covering the editing canvas until closed.
- **Anything Unusual About the Screen:** Allows full reference verification without context-switching to another page or browser tab.

## Screen 16: Proofreading Workspace

- **Screen Name:** Proofreading Workspace (Conservative Language & Theological Check)
- **Route / Path:** `currentView === 'sessions'`, `activeView === 'proofreading'`
- **Relevant Source File(s):** `frontend/src/components/proofreading/ProofreadingView.jsx`,  
`frontend/src/components/proofreading/ProofreadingChangesList.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Dedicated proofreading environment where the AI applies conservative grammatical, punctuation, and theological capitalization rules to the approved edited report, presents an itemized changes list, and allows human proofreaders to accept or adjust the text before final publication.
- **What Is Currently Displayed:**
- **Top Navigation Bar:**
- `← Back to Editing` button.

- Sermon Title, Minister, and Date.
- `□□ PROOFREADING PHASE` badge.
- Stepper: `1–6 [Done] → 7 [Active] → 8 [Future]`.
- **Action Header Banner:**
  - Before Run: Informational card with `□ Run AI Proofreader` primary button.
  - After Run: Green card with checkmark `□`, "Proofreading Review Complete: X adjustments identified", `□ Re-run Proofreading` button, and primary CTA `□ Accept & Complete Proofreading →`.
- **2-Column Proofreading Workspace:**
  - **Left Column (Proofreading Audit Panel):**
    - Switcher Tabs: `[ Identified Changes (X) | Edited Source ]`.
    - Changes Tab: Categorized list of all AI adjustments:
      - `Grammar & Punctuation` (original text vs suggested correction).
      - `Divine Capitalization` (e.g. capitalized divine pronouns: He, Him, His, Holy Spirit).
      - `Scripture Formatting` (standardized Book Chapter:Verse syntax).
      - `Terminology Consistency` (DLBC terms).
    - Edited Source Tab: Read-only view of the incoming edited report for side-by-side comparison.
  - **Right Column (Editable Proofread Document):**
    - Sermon Title form input.
    - Full ``<textarea>`` with proofread text. Proofreader can type manual adjustments directly.
    - Word and character counters.
    - `□ Save Adjustments` button.
- **Primary User Action:** Review identified changes, make final polish edits, and click `□ Accept & Complete Proofreading →`.
- **Secondary Actions:**
  - Toggle between Identified Changes and Edited Source tab.
  - Click `□ Save Adjustments` to snapshot a manual proofreading revision.
  - Click `Manage` on the proofreading standard badge to tune grammar and capitalization rules.
  - Click `← Back to Editing` to return to the editor workspace.
- **Navigation Available:** Back to editing, accept forward action to final report, standards modal.
- **Important Components Used:** `ProofreadingView`, `ProofreadingChangesList`, `ProofreadingStandardsModal`.
- **Modals, Sheets, Dropdowns:** Re-run confirmation modal; `ProofreadingStandardsModal`.
- **Empty States:** If not run yet: shows empty card with `□ Run AI Proofreader` CTA.
- **Loading States:** Animated banner: "AI Proofreader checking grammar, punctuation, and scripture references...".
- **Error States:** Red error banner if proofreading API fails.
- **Success/Confirmation States:** Banner confirms acceptance and unlocks final document generation.
- **Responsive / Mobile Behaviour:** Left changes panel and right text editor stack vertically on mobile. Reviewing changes while editing text requires extensive vertical scrolling.
- **Anything Unusual About the Screen:** Conservative non-rewrite policy: System prompts explicitly forbid the AI from rewriting sentences or changing the preacher's voice; it is strictly constrained to syntax, punctuation, capitalization, and typos.

## Screen 17: Proofreading Standards Modal

- **Screen Name:** Proofreading Standards Modal
- **Route / Path:** Modal dialog inside `ProofreadingView` or `SettingsView`
- **Relevant Source File(s):** `frontend/src/components/proofreading/ProofreadingStandardsModal.jsx`
- **Who Can Access It:** All operators.

- **Purpose of the Screen:** Allows proofreaders to inspect and configure the rules for punctuation, grammar, scripture citation style, and DLBC divine capitalization conventions.
- **What Is Currently Displayed:**
  - Modal Header: "Proofreading Standards Management", version tag ('v1'), close button 'X'.
  - Guidelines and Rules Textareas:
  - Proofreading Guidelines (grammar, syntax, tone retention).
  - Terminology Glossary (DLBC specific spelling and titles).
  - Formatting Rules (King James Version citation formats, divine pronoun capitalization).
  - Revision notes input.
  - Version History tab with rollback buttons.
  - Footer: 'Cancel' and 'Save & Activate Standard' buttons.
- **Primary User Action:** Edit formatting rules and click 'Save & Activate Standard'.
- **Secondary Actions:** Rollback to an earlier proofreading standard.
- **Navigation Available:** Close button 'X', backdrop click, Cancel button.
- **Important Components Used:** 'ProofreadingStandardsModal'.
- **Modals, Sheets, Dropdowns:** Modal overlay.
- **Empty States:** Version history empty state.
- **Loading States:** 'Saving...' button state.
- **Error States:** Validation error if guidelines are blank.
- **Success/Confirmation States:** Success notification on version save.
- **Responsive / Mobile Behaviour:** Responsive modal filling viewport on mobile.
- **Anything Unusual About the Screen:** Includes specific doctrinal capitalization rules (e.g. Saviour, General Superintendent, Scripture, Holy Ghost).

## Screen 18: Final Report Workspace & Word (.docx) Export

- **Screen Name:** Final Report Workspace & Export
- **Route / Path:** 'currentView === 'sessions'', 'activeView === 'final\_report''
- **Relevant Source File(s):** 'frontend/src/components/final\_report/FinalReportView.jsx'
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** The final destination of the sermon reporting pipeline. Finalizes the publication document, displays a formatted preview, and provides instant download of the official Microsoft Word (.docx) file.
- **What Is Currently Displayed:**
- **Top Floating Action Card:**
  - Green checkmark icon circle '✔'.
  - Tag: 'WORKFLOW COMPLETE'.
  - Title: "Final Report Ready".
  - Sermon subtitle.
  - Actions: 'Copy Text' button and prominent 'Download .docx' primary button.
- **Pre-Finalization State (if not finalized):**
  - Centered card with trophy '🏆', "Ready for Final Report Generation", explanation, suggested export filename (e.g. 'Sunday Worship Service - Message Report.docx'), and large button 'Finalize & Generate Word Document'.
- **Post-Finalization 2-Column Archival View:**
- **Left Column (Session Metadata & Post-Finalization Options):**
  - Session Metadata panel (Date, Service, Preacher, Duration, File size in KB).
  - 'Download Word Document (.docx)' primary button.
  - Secondary buttons: 'Edit Final Text' (unlocks editing) and 'Copy Clean Text'.

- Archival Record pill: `□ Saved to Persistent Storage`.
- **Right Column (Document Reading Canvas):**
  - Clean white paper-styled reading canvas rendering the complete finalized sermon report.
  - Structured typography: Title, Date, Speaker, Outline, Point 1, Point 2, Point 3, Sub-points, Scripture blocks, and Concluding Prayer.
  - **Primary User Action:** Click `Download .docx □` to download the official formatted Word document.
  - **Secondary Actions:**
    - Click `□ Copy Text` to copy plain formatted markdown to clipboard for WhatsApp broadcasting.
    - Click `□□ Edit Final Text` to make last-minute typo corrections and save as a new final revision.
    - Click `← Back` in the top bar to return to the Session Workspace hub.
  - **Navigation Available:** Top header back action, download links.
  - **Important Components Used:** `FinalReportView`, backend `python-docx` export generator.
  - **Modals, Sheets, Dropdowns:** Revision history modal for final reports.
  - **Empty States:** Pre-finalization card displayed before operator clicks Finalize.
  - **Loading States:** `Generating Document...` button state.
  - **Error States:** Red error banner if document generation or download fails.
  - **Success/Confirmation States:** Success banner: `□ Final Report finalized successfully! Microsoft Word (.docx) document is ready for download.`.
- **Responsive / Mobile Behaviour:**
  - Desktop: 2-column layout (Metadata left, Paper canvas right).
  - Mobile (390px): Stacks metadata above paper canvas. Download button stays prominent at the top.
- **Anything Unusual About the Screen:** The `.docx` download uses Python's `python-docx` engine on the backend, generating a document styled with DLBC official typographic headers, margins, and callout blocks suitable for immediate printing or church bulletin distribution.

## Screen 19: Transcribe Recorded File Pipeline

- **Screen Name:** Transcribe Recorded File Pipeline
- **Route / Path:** `currentView === 'transcribe'`
- **Relevant Source File(s):** `frontend/src/components/transcription/RecordedFileUploader.jsx`,  
`frontend/src/components/transcription/TranscriptionProgress.jsx`,  
`frontend/src/components/transcription/TranscriptsHistoryList.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Batch processing pipeline for uploading pre-recorded church audio files (WAV, MP3, M4A) or video files (MP4), choosing the transcription engine (Azure, Whisper, or Google), and monitoring progress.
- **What Is Currently Displayed:**
- **2-Column Layout:**
- **Left Column (Upload & Configuration):**
  - Transcription Engine Selector: Radio card options for:
    - Azure Speech (en-NG) [South Africa North cloud].
    - Local Faster-Whisper [Offline CPU INT8].
    - Google Cloud Speech-to-Text [Cloud credentials].
  - Media Type Toggle: `□ Audio (WAV, MP3, M4A)` vs `□ Video (MP4)`.
  - Drag-and-Drop File Dropzone with file details (Filename, File size in MB).
  - Primary Button: `□ Start Transcription`.
  - Progress Panel: Multi-stage progress indicators (`Uploading file...` → `Processing audio...` → `Transcribing speech...` → `Indexing segments...`).
- **Right Column (Transcription History):**

- List of previously uploaded file transcripts with duration, date, engine used, and `Open Transcript` button.
- **Primary User Action:** Select file, choose engine, and click `Start Transcription`.
- **Secondary Actions:** Select an existing transcript from the right-hand history list to open it directly.
- **Navigation Available:** Top header back button to Dashboard, left sidebar.
- **Important Components Used:** `RecordedFileUploader`, `TranscriptionProgress`, `TranscriptsHistoryList`, `useRecordedTranscription`.
- **Modals, Sheets, Dropdowns:** Native file picker dialog.
- **Empty States:** When no file is selected, dropzone displays prompt. History list displays empty state if 0 transcripts exist.
- **Loading States:** Progress bar showing upload progress (0–100%) and polling spinner during transcription.
- **Error States:** Progress panel displays red error box with `Retry` button if transcription fails.
- **Success/Confirmation States:** Automatically opens `RawTranscriptViewer` upon job completion.
- **Responsive / Mobile Behaviour:** 2 columns stack on mobile devices.
- **Anything Unusual About the Screen:** Video upload extracts audio server-side using `imageio-ffmpeg` without degrading audio clarity.

## Screen 20: YouTube Session Ingestion & Live Tab Capture

- **Screen Name:** YouTube Session Ingestion & Live Tab Capture
- **Route / Path:** `currentView === 'youtube'`
- **Relevant Source File(s):** `frontend/src/components/youtube/YouTubeSessionView.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Specialized pipeline for capturing sermons broadcast on YouTube, supporting both live stream tab-audio capture and server-side background transcription of past sermon recordings.
- **What Is Currently Displayed:**
- **Top Header:** "YouTube Session", subtitle, and "← Back to Dashboard" button.
- **YouTube URL Input Card:**
- Text field for pasting YouTube video or live broadcast URL.
- `Inspect URL` action button.
- Inline error alert if URL is invalid or blocked.
- **Inspected Video Metadata Card (Appears after URL analysis):**
- Video thumbnail image preview.
- Extracted YouTube video title and channel name.
- Duration badge (or `LIVE BROADCAST` badge).
- **Programme Assignment Dropdowns:**
- Programme / Event dropdown (e.g. Global Crusade with Kumuyi).
- Session / Section dropdown.
- Minister / Speaker text input.
- Custom Sermon Title input (pre-populated with YouTube video title).
- **Dual Ingestion Action Cards:**
- **Card 1: Live Tab Audio Capture (Recommended for Live Streams):**
- Description: Captures real-time audio from the browser tab playing YouTube with zero download blocks.
- Action: `Start YouTube Tab Capture`.
- **Card 2: Server-Side Background Transcription (For Past Videos):**
- Description: Downloads audio in background via `yt-dlp` and runs Azure Speech transcription.
- Action: `Start Recorded Transcription`.
- Active Job progress bar (`Downloading audio...`, `Transcribing...`, `Completed`).

- **Primary User Action:** Paste YouTube link, click `Inspect URL`, verify metadata, and click `Start YouTube Tab Capture` or `Start Recorded Transcription`.
- **Secondary Actions:** Edit minister name or sermon title before starting; click back button to return to dashboard.
- **Navigation Available:** Top header back button to dashboard, left sidebar.
- **Important Components Used:** `YouTubeSessionView`, `useAudioCapture` (tab audio mode via `getDisplayMedia`).
- **Modals, Sheets, Dropdowns:** Browser tab selection picker modal triggered by `getDisplayMedia({ audio: true, video: true })`.
- **Empty States:** Dual action cards remain hidden until a valid YouTube link is inspected.
- **Loading States:** `Inspecting URL...` spinner during oEmbed/metadata lookup.
- **Error States:** Red alert if video is private, geoblocked, or invalid.
- **Success/Confirmation States:** Clicking Tab Capture transitions to `LiveRecordingView` with `● YOUTUBE TAB AUDIO` badge.
- **Responsive / Mobile Behaviour:** Metadata card and action cards stack on mobile.
- **Anything Unusual About the Screen:** Tab Audio Capture bypasses YouTube bot detection and bot-blocking entirely by using the browser's native tab-sharing audio loopback.

## Screen 21: Settings & AI Standards Hub

- **Screen Name:** Settings & AI Standards Hub
- **Route / Path:** `currentView === 'settings'`
- **Relevant Source File(s):** `frontend/src/components/settings/SettingsView.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Master administration center for managing church programmes, editing AI guidelines across all three pipeline stages, inspecting transcription engine configurations, and verifying system health.
- **What Is Currently Displayed:**
- **Header Bar:** Title "System Settings & AI Standards", subtitle, `Checked at [Time]` label, and `🔄 Refresh Status` button.
- **Section 1: Church Programmes & Sessions (Embedded Component):**
- Full programme directory (see Screen 22).
- **Section 2: AI Standards & Pipeline Instructions:**
- Stage Selector Tabs: `[ 📄 1. Reporting | 📄 2. Editing | 📄 3. Proofreading ]`.
- **Active Reporting Tab:**
- Active version pill (`Active: Reporting Standard v1`) and `📄 Version History` button (opens modal).
- Editable Textareas:
- General Reporting Guidelines.
- Reporter A Instructions (Thematic structure).
- Reporter B Instructions (Details & omissions).
- Church Terminology & Glossary.
- Approved Few-Shot Examples.
- Change Notes input.
- Primary Action: `📄 Save & Activate Reporting Standard`.
- **Active Editing Tab:**
- Active version pill (`Active: Editor Standard v1`) and `📄 Version History` button.
- Editable Textareas: General Guidelines, Compilation Guidance, Terminology, Approved Examples, Change Notes.
- Primary Action: `📄 Save & Activate Editor Standard`.
- **Active Proofreading Tab:**
- Active version pill (`Active: Proofreading Standard v1`) and `📄 Version History` button.
- Editable Textareas: Proofreading Guidelines, Terminology Glossary, Formatting Rules (Divine pronouns, Scripture citation), Change Notes.

- Primary Action: `□ Save & Activate Proofreading Standard`.
- **Section 3: Transcription Providers & System Status:**
- Status Cards Grid:
- Azure Speech: Status (`□ CONFIGURED & READY`), region (`southafricanorth`), language (`en-NG`).
- Faster-Whisper: Status (`□ AVAILABLE (OFFLINE)`), model (`small.int8`), device (`cpu`).
- Google Cloud Speech: Status (`□ NOT CONFIGURED` or ready).
- Backend Server: Host (`http://localhost:8000` or production URL), database status (`SQLite / app.db`).
- **Primary User Action:** Update prompt instructions or terminology and click `Save & Activate Standard`.
- **Secondary Actions:** Switch between pipeline tabs; manage church programmes; open version history modals; click Refresh.
- **Navigation Available:** Topbar back button to Dashboard, left sidebar.
- **Important Components Used:** `SettingsView`, `ProgrammesSettingsSection`, `ReportingStandardsModal`, `EditorStandardsModal`, `ProofreadingStandardsModal`.
- **Modals, Sheets, Dropdowns:** Three separate standards version history modals.
- **Empty States:** None.
- **Loading States:** Initial load fetches status from 7 backend endpoints simultaneously with spinner.
- **Error States:** Red connection banner if backend is unreachable.
- **Success/Confirmation States:** Green confirmation message on saving new standard versions.
- **Responsive / Mobile Behaviour:**
- Desktop: Clean tabs, spacious form controls.
- Mobile (390px): Multi-line tab bar, long scrollable forms.
- **Anything Unusual About the Screen:** Direct prompt engineering in the UI: Editors do not need developer assistance to update how Gemini summarizes or proofreads sermons.

## Screen 22: Church Programmes & Sessions Management

- **Screen Name:** Church Programmes & Sessions Management
- **Route / Path:** Embedded section inside `SettingsView`
- **Relevant Source File(s):** `frontend/src/components/settings/ProgrammesSettingsSection.jsx`
- **Who Can Access It:** All operators.
- **Purpose of the Screen:** Allows the Information Unit to define church programmes (e.g. Sunday Worship Service, Monday Bible Study, December Retreat) and configure child sections (e.g. Morning Message, Faith Clinic, Youth Section) that populate all dropdowns throughout the app.
- **What Is Currently Displayed:**
- **Header:** Title "Church Programmes & Sections", subtitle, `+ Add Programme` button, and `Show Archived` toggle switch.
- **Add Programme Form (Collapsible):** Name input, `Save Programme` button, `Cancel` button.
- **Programmes Accordion List:**
- Each programme item displays:
- Expand/Collapse chevron `□` / `▼`.
- Programme Name with `□□ Rename` and `□ Archive` actions.
- Active session count badge (`X Sections`).
- **Expanded Programme Body (Child Sessions):**
- List of configured sections under this programme.
- Each section has inline `□□ Rename` and `□ Archive` buttons.
- Inline `+ Add Section / Session` form with name input and `Add` button.
- **Primary User Action:** Click `+ Add Programme` or expand a programme to add a new section.

- **Secondary Actions:** Rename programmes or sections; archive/unarchive outdated programmes; toggle 'Show Archived'.
- **Navigation Available:** Embedded in Settings; no external page navigation.
- **Important Components Used:** 'ProgrammesSettingsSection'.
- **Modals, Sheets, Dropdowns:** Inline forms (no popover dialogs).
- **Empty States:** If 0 programmes exist: "No programmes configured yet. Click '+ Add Programme' to begin."
- **Loading States:** Loading indicator while fetching from '/api/programmes'.
- **Error States:** Inline error notice if name is duplicate or blank.
- **Success/Confirmation States:** Green notification pill: '☑ Programme created successfully'.
- **Responsive / Mobile Behaviour:** Accordions work smoothly on mobile; action buttons wrap cleanly.
- **Anything Unusual About the Screen:** Cascading deletion safeguard: Archiving a programme safely hides its sessions from creation dropdowns without breaking historical sessions in the database.

### 3. USER FLOW MAP

This section documents all primary and secondary workflows currently implemented in the application. For each flow, the step-by-step path, decision points, state transitions, refresh resilience, and friction points are detailed from actual source code execution.

#### Flow 1: Live Audio Recording & Processing (Primary Live Workflow)

- **Trigger:** Operator needs to record and transcribe a live church service or programme.
- **Start Screen:** Home Dashboard ('DashboardView.jsx') -> clicks 'Start Live Session' primary button (or selects "New Live Recording" from sidebar).
- **Step-by-Step Path:**
- **Session Configuration ('NewLiveSessionView'):**
  - Operator selects a Church Programme from the dropdown (e.g. "Sunday Worship Service"). Sessions dropdown populates with active sessions under that programme.
  - Operator selects Session (or types custom title into Title field).
  - Operator selects Minister / Speaker from dropdown or enters custom name (e.g. "Pastor W.F. Kumuyi").
  - Operator inputs Message / Sermon Title (e.g. "Walking in the Light of Divine Grace").
  - Operator selects Audio Input Device from dropdown ('AudioSourceSelector.jsx'). Audio level meter animates live, verifying microphone signal.
  - Operator selects AI Transcription Provider (Radio button: 'Azure Speech (Recommended)' or 'Faster-Whisper (Offline)').
- **Initiate Recording:**
  - Operator clicks 'Start Live Session & Recording'.
  - Browser prompts for microphone permission (if not already granted).
  - MediaStream initializes with 16kHz mono audio constraints.
  - Hook 'useAudioCapture.js' calls 'POST /api/sessions/live' with metadata, creating a persistent session in SQLite with status 'recording'.
  - Azure Speech Recognizer or local Whisper worker connects.
- **Live Monitoring ('LiveRecordingView'):**
  - App transitions to full-screen recording monitor.
  - Timer increments in 'HH:MM:SS' (tabular numerals).
  - Live transcript chunks stream into the live transcript card in real-time.
  - Audio level meter visualizes input dBFS.
  - Operator can click '🚩 Flag for Verification' at any time to bookmark an ambiguous term or Scripture citation for post-session review.

- **Stop Recording:**

- Operator clicks `□ Stop Session & Process`.
- Confirmation modal warns: "Are you sure you want to conclude this live recording?". Operator confirms.
- Audio stream is finalized and committed to `.wav` file on backend.
- Backend finalizes transcript, calculates word count and duration, runs automatic phoneme / church glossary flag detection, and updates session status to `completed`.

- **Completion Summary Modal (`SessionCompletionView`):**

- Modal pops up displaying total duration, words captured, and flags detected.
- Operator chooses next action: `Begin Verification Now`, `Skip to Reporting`, `View Session Details`, or `Finish for Now`.

- **Decision Points:**

- Flags > 0 vs Flags == 0: If 0 flags are detected, operator can click `Skip to Reporting` which automatically calls `POST /api/sessions/{id}/confirm-raw-as-verified` and skips manual verification.
- Azure vs Whisper: Azure requires active internet connection; Whisper runs completely local.
- **Success State:** Session saved in SQLite with `.wav` audio, segment array, and initial verification items; operator lands on Session Workspace or Verification screen.

- **Error / Failure Paths:**

- Microphone access denied: Error banner shows browser permission prompt instructions.
- Azure WebSocket drop: App falls back to local buffer; reconnection alert displayed.
- Backend connection timeout: Audio continues recording locally in browser memory until backend recovers.

- **Page Refresh (F5) Resilience:**

- **DANGER:** Live recording state is held in React state and in-memory Web Audio MediaStream. Pressing F5 during live recording terminates the audio stream abruptly. Backend marks session as `interrupted`. Upon re-opening the session, the Session Workspace shows an "Interrupted Session" incident banner offering one-click recovery.

- **Bottlenecks / Stuck Points:**

- If microphone permission is blocked at OS level, the start button hangs until timeout.
- If Azure network latency spikes, the live transcript lag creates operator anxiety that recording failed.

## Flow 2: Floating Recorder & Multitasking Workflow

- **Trigger:** While live recording is running, the operator needs to check past transcripts, verify a previous sermon, or look up settings without stopping the current recording.
- **Start Screen:** `LiveRecordingView` (Full-screen active recorder).
- **Step-by-Step Path:**
  - Operator clicks `Minimize Recorder` (or clicks any navigation link in the AppShell sidebar).
  - The full-screen monitor unmounts, and the chosen view (e.g. Dashboard, Sessions History, Settings) renders.
  - `FloatingRecordingController.jsx` appears pinned to the bottom-right corner of the viewport (Z-index 9999).
  - Floating widget displays: Pulsing red recording dot, session title, elapsed timer, live audio level bar, and latest live transcript snippet.
  - Operator navigates between any screens, reviews past reports, or edits settings.
  - Operator clicks `□ Expand` on the floating widget: App immediately restores full-screen `LiveRecordingView`.
  - Alternatively, operator clicks `□ Stop` directly on the floating widget: Recording concludes and triggers the Session Completion Modal.
- **Decision Points:** Expand back to full screen vs stop recording directly from widget.
- **Success State:** Audio capture continues unbroken in background while multitasking across the system.
- **Error / Failure Paths:** Heavy CPU operations (e.g. running local Whisper transcription on another file) can cause Web Audio buffer underruns on lower-end laptops.
- **Page Refresh (F5) Resilience:** Hard refresh terminates recording.

- **Bottlenecks / Stuck Points:** Floating widget can overlap primary action buttons at the bottom-right of certain tables or modals if screen height is constrained.

## Flow 3: Recorded File Upload & Transcription

- **Trigger:** User has an existing audio or video file (WAV, MP3, M4A, MP4) from an external recorder or broadcast team and needs to transcribe it.
- **Start Screen:** `DashboardView` dropzone or `Transcribe Recording File` sidebar item (`RecordedFileUploader.jsx`).
- **Step-by-Step Path:**
  - Operator drags-and-drops or clicks to browse file (`RecordedFileUploader`).
  - App parses file metadata (name, size, MIME type) and displays file chip.
  - Operator selects provider (`Azure Speech` or `Faster-Whisper`).
  - Operator clicks `Upload & Transcribe`.
  - `TranscriptionProgress.jsx` renders:
    - Stage 1: Uploading file to `/api/transcribe/upload` (animated percentage bar).
    - Stage 2: Processing / transcribing (polling `/api/transcribe/job/{id}` status every 1.5 seconds).
  - On job completion, app renders `RawTranscriptViewer.jsx` in full-screen mode with built-in audio player and synchronized word timestamps.
  - The new transcript appears in `TranscriptsHistoryList.jsx` right sidebar for future retrieval.
- **Decision Points:** Provider selection (Azure for high cloud accuracy vs Faster-Whisper for zero-data offline privacy).
- **Success State:** Full transcript generated with timestamped segments and playable audio scrubber.
- **Error / Failure Paths:**
  - File size exceeds server limit (500MB): Upload rejected with error banner.
  - Unsupported format: Pre-flight check alerts operator before network upload starts.
  - Job fails midway: Operator presented with `Retry Transcription` button.
- **Page Refresh (F5) Resilience:** Upload progress resets to idle. Completed jobs remain in backend SQLite and can be reopened from `TranscriptsHistoryList`.
- **Bottlenecks / Stuck Points:** Long sermons (2+ hours) can take several minutes on local Faster-Whisper without a dedicated GPU. Polling progress bar lacks ETA estimation.

## Flow 4: YouTube Ingestion & Tab Capture

- **Trigger:** Church service is broadcasting live or archived on YouTube and needs transcription.
- **Start Screen:** `YouTubeSessionView.jsx` (accessible from Dashboard or sidebar).
- **Step-by-Step Path:**
  - **Mode A: Live Chrome Tab Audio Capture:**
    - Operator enters Programme, Session, and Sermon metadata.
    - Operator clicks `Start Tab Capture & Transcribe`.
    - Browser displays native Chrome tab picker dialog: "Choose what to share".
    - Operator selects the YouTube tab and checks **"Share tab audio"**.
    - App captures the raw digital audio stream from the browser tab and begins real-time transcription, redirecting to `LiveRecordingView`.
  - **Mode B: YouTube URL Direct Ingestion (Archived Video):**
    - Operator pastes YouTube video URL (e.g. `https://youtube.com/watch?v=...`).
    - Operator clicks `Fetch Video Info`.
    - Backend extracts title, duration, and thumbnail.
    - Operator clicks `Download Audio & Transcribe`.
    - Backend downloads audio stream via `yt-dlp` and feeds it into the transcription engine.

- **Decision Points:** Live tab capture (for ongoing broadcasts) vs URL download (for already finished videos).
- **Success State:** New session created in church session directory with video metadata attached.
- **Error / Failure Paths:**
  - User forgets to check "Share tab audio" in Chrome modal: App detects silent audio stream and prompts operator: "No audio detected. Ensure 'Share tab audio' was checked."
  - YouTube age-restricted or copyright-blocked stream: Backend `yt-dlp` returns error message displayed in `ErrorBanner`.
- **Page Refresh (F5) Resilience:** Tab capture is terminated on refresh.
- **Bottlenecks / Stuck Points:** Chrome tab sharing permission requires 3 manual clicks by the user in the browser dialog.

## Flow 5: Verification & Audio Scrubbing Workflow

- **Trigger:** Session has concluded and contains ambiguous words, proper names, or flagged Scripture citations requiring human confirmation.
- **Start Screen:** `SessionDetailView` -> Verification tile or `VerificationWorkflow.jsx`.
- **Step-by-Step Path:**
  - Operator opens Session Workspace and clicks `Start Verification` (or `Review Flags` button).
  - Verification Workspace renders with:
    - Top Audio Scrubbing Player (`CompletedRecordingPlayer.jsx`).
    - Progress Bar showing `X of Y items resolved (Z%)`.
    - List of flagged items (`VerificationItemRow.jsx`) sorted chronologically.
  - Operator selects first flagged item:
    - Audio player immediately jumps to 2.5 seconds before the flagged timestamp and loops the 5-second audio snippet.
  - Row highlights in blue, showing flag reason (e.g. "Low Confidence (64%)", "Unfamiliar Term: 'Melchizedek'", "Scripture Mismatch").
  - Inline correction input auto-focuses with the current transcription text.
  - Operator confirms or corrects:
    - If correct: Operator clicks `Confirm Text` (or presses Enter).
    - If incorrect: Operator types correct biblical term/spelling and clicks `Apply Correction`.
    - If extraneous noise: Operator clicks `Discard / Mark False Positive`.
  - Row changes state to `Resolved` (green badge) and audio advances to next flagged item.
  - Operator can click `+ Add Manual Verification Point` anywhere in the transcript.
  - When all items are reviewed, operator clicks `Finalise & Approve Transcript`.
- **Decision Points:**
  - Individual item resolution vs `Confirm All Remaining` (bulk approval if operator is satisfied).
- **Success State:** All items resolved; session status transitions to `verified`; Verified Transcript artifact tile unlocks.
- **Error / Failure Paths:**
  - Audio file missing on disk: Audio player displays warning; operator can still manually correct text using transcript context.
- **Page Refresh (F5) Resilience:** Each resolved item is immediately saved to `verification\_items` table in SQLite via `PUT /api/verification/items/{id}`. Refreshing the browser preserves all resolved work.
- **Bottlenecks / Stuck Points:** No global keyboard shortcuts (e.g. Space to play/pause, Tab to next flag) forces heavy mouse clicking between audio controls and inputs.

## Flow 6: Dual Reporting Generation & Comparison (Stage 4)

- **Trigger:** Verified transcript is approved; operator needs to generate structured sermon reports.
- **Start Screen:** `SessionDetailView` -> Stage 4: Reporting tile (`ReportingView.jsx`).
- **Step-by-Step Path:**

- Operator enters Reporting Workspace.
- Banner displays active AI Reporting Standard (`Reporting Standard v1`) with button to open `ReportingStandardsModal` if custom prompt adjustments are needed for this sermon.
- Operator clicks `□ Generate Dual Reports with Gemini`.
- App triggers parallel generation:
- `Reporter A` generates thematic, point-by-point, outline-oriented summary.
- `Reporter B` generates comprehensive, narrative-driven summary with detailed illustrations and verbatim pastoral quotes.
- Dual-column comparative layout renders side-by-side:
- Column 1: Reporter A draft (`ReportCard.jsx`).
- Column 2: Reporter B draft (`ReportCard.jsx`).
- Each card displays: Word count, generation timestamp, model name (`gemini-2.5-flash`), markdown formatting, and copy button.
- Operator reviews both perspectives to ensure theological accuracy and no missed sermon points.
- Operator clicks `Proceed to Editing Stage →`.
- **Decision Points:**
- Regenerate individual reporter draft if Gemini hallucinated or truncated.
- Edit active reporting standard prompt before triggering generation.
- **Success State:** Both reports generated and stored in SQLite `reports` table; Editing stage unlocked.
- **Error / Failure Paths:**
- Gemini API quota/rate limit error: Inline error banner with exponential backoff retry button.
- Truncated output: Operator can trigger single-column regeneration.
- **Page Refresh (F5) Resilience:** Generated reports are persistently saved in database. Refreshing preserves drafts.
- **Bottlenecks / Stuck Points:** Side-by-side columns on screens narrower than 1200px wrap vertically, making direct comparison harder without scrolling.

## Flow 7: Editorial Synthesis & Source Reference Flyout (Stage 5)

- **Trigger:** Dual reports exist; Chief Editor needs to synthesize them into a unified, authoritative draft.
- **Start Screen:** `EditingView.jsx` (accessible from Reporting view or Session Workspace).
- **Step-by-Step Path:**
- Editor enters Editing Workspace.
- Workspace presents an editorial workbench:
- Left / Main Area: Rich markdown editor populated with synthesized draft (or blank with `Generate Unified Synthesis Draft` button).
- Top Action Bar: Active Editor Standard badge, `View Standards` button, Word count, Synthesis button.
- Top Right Toggle: `□ Open Source Reference Flyout` button.
- Editor clicks `Open Source Reference Flyout`:
- Slide-out drawer (`SourceReferenceDrawer.jsx`) smoothly animates from the right edge of the screen.
- Drawer provides tabbed access to:
- Tab 1: `Reporter A Draft`
- Tab 2: `Reporter B Draft`
- Tab 3: `Verified Full Transcript` (with search filter).
- Editor reviews sources while typing directly in the editor, ensuring every sermon illustration and scripture cross-reference is represented accurately.
- Editor can click `Save Draft` at any time (status: `in\_review`).
- When complete, editor clicks `□ Approve & Finalise Editorial Draft`.
- **Decision Points:**

- AI automated synthesis vs manual operator writing.
- Drawer open/closed toggle based on screen real estate.
- **Success State:** `edited\_reports` row created/updated; editing status marked `complete`; Proofreading stage unlocked.
- **Error / Failure Paths:** Network failure during auto-save: Local copy preserved in component state; alert warns before closing.
- **Page Refresh (F5) Resilience:** Unsaved changes in textarea are lost on refresh; saved drafts persist in SQLite.
- **Bottlenecks / Stuck Points:** Textarea is plain text/markdown without WYSIWYG rich text formatting controls (no bold/italic toolbar buttons).

## Flow 8: Proofreading & Changes Acceptance Workflow (Stage 6)

- **Trigger:** Editorial synthesis draft is approved and requires grammatical, doctrinal, and stylistic proofreading.
- **Start Screen:** `ProofreadingView.jsx`.
- **Step-by-Step Path:**
  - Proofreader enters Proofreading Workspace.
  - System checks active Proofreading Standards (`ProofreadingStandardsModal.jsx` rules: King James citation formatting, Divine pronoun capitalization e.g. "He", "Him", "His", church title nomenclature).
  - Proofreader clicks `□ Run AI Proofreading Inspection`.
  - Backend runs Gemini proofreading analysis comparing source editorial draft against style standards.
  - View renders two synchronized panes:
    - Left Pane: Marked-up text showing inline insertions (green highlight) and deletions (red strikethrough).
    - Right Pane: List of distinct proposed changes (`ProofreadingChangesList.jsx`), grouped by category (Grammar, Doctrinal Terminology, Divine Pronoun, Scripture Reference).
  - Proofreader evaluates each change:
    - Click `□ Accept` on an individual item: Change is committed to final draft.
    - Click `□ Reject`: Original editorial wording is restored.
    - Click `□ Accept All High-Confidence Changes`: Commits all grammatical corrections in one click.
  - Proofreader reviews resulting clean text.
  - Proofreader clicks `Finalise Proofread Text & Send to Final Report`.
- **Decision Points:** Granular line-by-line acceptance vs bulk acceptance.
- **Success State:** Proofread text finalized and saved in `proofread\_reports` table; Final Report stage unlocked.
- **Error / Failure Paths:** If Gemini returns invalid diff format, the system falls back to standard text comparison and preserves the editorial draft intact.
- **Page Refresh (F5) Resilience:** Proposed changes and accepted statuses persist in database.
- **Bottlenecks / Stuck Points:** On mobile or narrow tablet viewports, the side-by-side diff collapses into a vertical stack requiring long scrolls.

## Flow 9: Final Report & Word .docx Export (Stage 7)

- **Trigger:** Proofreading is approved; Information Unit needs to export the publication-ready bulletin/report for church leaders and media distribution.
- **Start Screen:** `FinalReportView.jsx`.
- **Step-by-Step Path:**
  - Operator enters Final Report Workspace.
  - Clean, beautifully styled document preview displays:
    - Official Church Header & Logo placeholder.
    - Session Title, Programme, Date, Minister, Scripture Texts.
    - Executive Thematic Outline.

- Full Polished Sermon Transcript & Synthesis.
- Key Takeaways & Pastoral Applications.
- Operator can make last-minute inline text adjustments in the editable preview.
- Operator selects export action:
- `Download Word Document (.docx)`: Backend generates formatted Microsoft Word `.docx` file using `python-docx` with standard church styling, headings, headers, footers, and margins.
- `Copy Polished Text to Clipboard`: Copies markdown/plain text formatted for WhatsApp or email blasts.
- `Print / Save as PDF`: Triggers browser print preview with `@media print` clean CSS.
- **Decision Points**: Export as `.docx` vs copy text vs print.
- **Success State**: Word document downloaded; session marked `complete` across entire pipeline.
- **Error / Failure Paths**: Backend docx generation error: Clear error banner with fallback copy button.
- **Page Refresh (F5) Resilience**: Completed report remains permanently accessible in Sessions History.
- **Bottlenecks / Stuck Points**: No built-in PDF export button (relies on browser Print-to-PDF).

## Flow 10: Church Programmes & Sessions Configuration

- **Trigger**: Operator needs to add a new church convention, retreat, or weekly service type to dropdowns.
- **Start Screen**: `SettingsView.jsx` -> `Church Programmes & Sections` accordion.
- **Step-by-Step Path**:
- Operator clicks `Settings & Standards` in sidebar.
- Under Programmes section, clicks `+ Add Programme`.
- Enters programme name (e.g. "Dec 2026 National Retreat").
- Clicks `Save Programme`. Programme card appears in accordion.
- Operator expands the new programme and enters child session name (e.g. "Day 1 - Evening Revival Service").
- Clicks `Add Section / Session`.
- Newly created programmes immediately populate dropdowns in `NewLiveSessionView`, `DashboardView`, and `YouTubeSessionView`.
- **Decision Points**: Rename existing vs Archive obsolete programme.
- **Success State**: Updated database records in `programmes` and `programme_sessions` tables.
- **Error / Failure Paths**: Duplicate name validation displays red warning.
- **Page Refresh (F5) Resilience**: Immediate persistence in SQLite.
- **Bottlenecks / Stuck Points**: Cannot drag-and-drop to reorder programmes.

## Flow 11: Prompt Guidelines & AI Standards Management

- **Trigger**: Editorial committee updates reporting guidelines, terminology glossary, or theological phrasing rules.
- **Start Screen**: `SettingsView.jsx` -> AI Standards & Pipeline Instructions tabs.
- **Step-by-Step Path**:
- Operator clicks `Settings & Standards` in sidebar.
- Selects stage tab: `1. Reporting`, `2. Editing`, or `3. Proofreading`.
- Modifies prompt fields: General guidelines, Reporter A instructions, Reporter B instructions, Terminology glossary, or Few-shot examples.
- Enters change note (e.g. "Added rules for divine pronouns in Genesis sermon").
- Clicks `Save & Activate Standard`.
- Backend creates a new versioned row in `reporting_standards`, `editor_standards`, or `proofreading_standards`, setting it as `is_active = 1`.
- Operator clicks `Version History` to view past versions or roll back.
- **Decision Points**: Save as new active version vs inspect previous versions.

- **Success State:** Future AI generations automatically use the new prompt instructions.
- **Error / Failure Paths:** Empty required field blocks submission with inline validation notice.
- **Page Refresh (F5) Resilience:** Immediate persistence.
- **Bottlenecks / Stuck Points:** No prompt preview or test-run sandbox before saving.

## Flow 12: Interruption Recovery & Operational Safety

- **Trigger:** Computer unexpectedly lost power, browser crashed, or laptop lid closed during a live recording.
- **Start Screen:** `DashboardView` or `SessionHistoryList`.
- **Step-by-Step Path:**
  - Operator reopens application.
  - The crashed session appears in Recent Sessions with status badge `INTERRUPTED` (amber).
  - Operator clicks the interrupted session to open `SessionDetailView`.
  - `SessionDetailView` renders a high-visibility Operational Incident Banner:
    - "🔴 This session was interrupted unexpectedly. 42 minutes of audio and 3,120 transcript words were recovered safely."
  - Incident Banner provides two recovery actions:
  - `Resume Processing`: Finalizes the recovered audio file and runs flag detection, advancing the session to `completed` and unlocking verification.
  - `Mark as Concluded`: Seals the session as-is.
  - Operator clicks `Resume Processing`.
  - App repairs session metadata and unlocks normal pipeline progression.
- **Decision Points:** Resume processing vs discard incomplete recording.
- **Success State:** Partial recording saved losslessly; operator can proceed through verification and reporting.
- **Error / Failure Paths:** Corrupted audio file: System preserves raw text segments and marks audio unavailable.
- **Page Refresh (F5) Resilience:** Incident banner remains until explicitly resolved.
- **Bottlenecks / Stuck Points:** If operator doesn't notice the banner, they may wonder why downstream stages are locked.

# 4. NAVIGATION STRUCTURE & SITEMAP

## Navigation Architecture Overview

The DLBC Information Unit App uses a unified desktop-first layout powered by `AppShell.jsx`. There is **no client-side router** (such as React Router, TanStack Router, or Wouter); view switching is managed entirely via top-level React state (`currentView` in `App.jsx`) and local tab state (`activeView` in `SessionDetailView.jsx`).

|                                     |                                                    |                  |
|-------------------------------------|----------------------------------------------------|------------------|
| +-----+                             |                                                    |                  |
| APP HEADER (AppShell.jsx)           |                                                    |                  |
| [Brand Dot + DLBC Information Unit] | [Screen Title]                                     | [Actions / User] |
| +-----+                             |                                                    |                  |
| SIDEBAR NAVIGATION                  | MAIN CONTENT VIEWPORT                              |                  |
|                                     |                                                    |                  |
| [+ Start Live] (CTA)                | (Switches based on currentView):                   |                  |
| * Dashboard                         | - DashboardView.jsx                                |                  |
| * Sessions History                  | - SessionDetailView.jsx (Sub-view routing)         |                  |
| * New Live Session                  | - LiveRecordingView.jsx (Full-screen monitor)      |                  |
| * Transcribe File                   | - RecordedFileUploader.jsx / Progress / Transcript |                  |
| * YouTube Session                   | - YouTubeSessionView.jsx                           |                  |
| * Settings & Rules                  | - SettingsView.jsx                                 |                  |

```
| | |
| [Bottom System Pill] | |
+-----+
| [FLOATING RECORDER CONTROLLER (Z-index 9999, rendered when recording & minimized)]|
+-----+
```

Sidebar Navigation Items & Badges

- **Start Live Session (Primary Button CTA):**
  - Distinct blue pill at top of sidebar.
  - If a recording is already active, clicking this expands the active recording instead of opening a duplicate.
- **Dashboard (‘dashboard’):** Icon: ‘□’. Main landing screen with system KPIs, quick actions, audio dropzone, and recent sessions table.
- **Sessions History (‘sessions’):** Icon: ‘□’. Full searchable and filterable directory of all past church sessions.
- **New Live Session (‘new\_live’):** Icon: ‘□□’. Pre-session setup form for live microphone recording.
- **Transcribe File (‘transcribe’):** Icon: ‘□’. Audio/video file upload, progress monitor, and raw transcript reader.
- **YouTube Session (‘youtube’):** Icon: ‘□□’. Live Chrome tab audio capture and YouTube URL ingestion.
- **Settings & Standards (‘settings’):** Icon: ‘□□’. Church programmes manager and AI prompt standards editor.

Contextual Topbar & Back Button Mechanics

The header bar in ‘AppShell.jsx’ dynamically computes its title and back button based on current application state:

- **Title Resolution:**
  - On top-level views: Displays view name (e.g. "Dashboard", "Sessions History", "Settings & Standards").
  - In Session Detail Sub-Views: Communicated upwards via callback ‘onSubViewChange’:
    - Overview: "Session Workspace"
    - Verification: "Verification"
    - Reporting: "Reporting"
    - Editing: "Editing"
    - Proofreading: "Proofreading"
    - Final Report: "Final Report"
- **Back Button (‘← Back’):**
  - Visible whenever ‘currentScreenBack’ is a valid function.
  - On sub-views inside ‘SessionDetailView’: Returns to ‘Session Workspace’ (overview).
  - On ‘SessionDetailView’ overview: Returns to ‘Sessions History’ list.
  - On ‘NewLiveSession’, ‘Transcribe’, ‘YouTube’, or ‘Settings’: Returns to ‘Dashboard’.
- **Browser Back Button Warning:** Because there is no browser URL routing, pressing the browser’s native Back button (‘Alt + ←’ or mouse back) navigates away from the entire web app or reloads the default route, losing temporary form state.

Complete Hierarchical Text Sitemap

```
App (Root)
|
├─ AppShell (Master Layout Container)
|   └─ Top Header (Brand, Dynamic Title, In-App Back Button, Live Status Pill)
|   └─ Left Sidebar (Navigation links, Start Live Session CTA, System status)
|   |
```

- | | | [View 1] DashboardView (Route: &#x27;dashboard&#x27;)
- | | | | Metric KPI Cards (Active Sessions, Total Hours, Needs Verification, Completed)
- | | | | Quick Action Cards (New Live Recording, Transcribe Recorded File, YouTube Session)
- | | | | Direct File Dropzone (Drag-and-drop audio for instant transcription)
- | | | | Recent Sessions Table (Last 10 sessions with status badges and open links)
- | | |
- | | | [View 2] NewLiveSessionView (Route: &#x27;new\_live&#x27;)
- | | | | Programme & Section Selectors (Populated from SQLite)
- | | | | Minister & Message Title Inputs
- | | | | Audio Input Selector (Microphone picker + live dBFS AudioLevelMeter)
- | | | | Transcription Provider Radio Group (Azure Speech vs Faster-Whisper)
- | | | | Action Buttons (Start Live Session & Cancel)
- | | |
- | | | [View 3] LiveRecordingView (Route: &#x27;live\_recording&#x27; - Full Screen Monitor)
- | | | | Live Session Header (Session Title, Programme, Speaker)
- | | | | Big Elapsed Timer Display (Tabular numerals HH:MM:SS)
- | | | | Audio Input Signal VU Meter (AudioLevelMeter.jsx)
- | | | | Live Streaming Transcript Card (Auto-scrolling incoming words)
- | | | | Bottom Control Bar:
  - | | | | | Flag for Verification Button (Bookmarks timestamp)
  - | | | | | Minimize Recorder Button (Switches to floating controller)
  - | | | | | Stop Session & Process Button (Opens confirmation dialog)
- | | |
- | | | [Floating Widget] FloatingRecordingController (Active when recording minimized)
- | | | | Pulsing Recording Indicator & Elapsed Time
- | | | | Mini Live Transcript Snippet
- | | | | Mini Audio Level Bar
- | | | | Expand to Full Screen Button
- | | | | Stop Recording Button
- | | |
- | | | [Modal] SessionCompletionView (Appears after stopping live recording)
- | | | | Session Summary Stats (Duration, Words captured, Flags detected)
- | | | | Begin Verification Button
- | | | | Skip to Reporting Button (If flags == 0)
- | | | | View Session Workspace Button
- | | | | Finish for Now Button
- | | |
- | | | [View 4] SessionHistoryList (Route: &#x27;sessions&#x27; - No active session selected)
- | | | | Filter & Search Bar (Text search, Programme filter, Status tabs: All, Needs Verification, Completed)
- | | | | Sessions Data Table (Date, Title, Programme, Duration, Words, Flags, Status, Actions)
- | | | | Pagination / Load More Controls
- | | | | Delete Session Confirmation Modal
- | | |
- | | | [View 5] SessionDetailView (Route: &#x27;sessions&#x27; - Active session selected)
- | | | |
- | | | | Sub-View 5.0: Session Workspace Hub (Default Overview)
  - | | | | | Session Identifier & Metadata Bar (Inline editable title)
  - | | | | | Authoritative 8-Stage Lifecycle Stepper (Audio -> Final Report)
  - | | | | | Action Required Hero Banner (Contextual immediate next action)
  - | | | | | Operational Incident Banner (If session was interrupted)
  - | | | | | 6 Artifact Tiles Grid:

- | | | | └─ Tile 1: Master Audio Recording (Audio player, duration, download WAV)
- | | | | └─ Tile 2: Raw Speech-to-Text Transcript (Word count, view raw)
- | | | | └─ Tile 3: Verified Transcript (Flags count, start/view verification)
- | | | | └─ Tile 4: Reporter Drafts (Reporter A & B, open dual reporting)
- | | | | └─ Tile 5: Chief Editor Synthesis (Synthesis status, open editing)
- | | | | └─ Tile 6: Final Publication-Ready Report (Download Word, view final)
- | | | └─ Future Stages Roadmap Sidebar
- | | |
- | | └─ Sub-View 5.1: RawTranscriptViewer (&#x27;raw\_transcript&#x27;)
- | | | └─ Synchronized Audio Player Scrubber
- | | | └─ Word Timestamp Inspection Grid
- | | | └─ Export Raw Text Button
- | | |
- | | └─ Sub-View 5.2: VerificationWorkflow (&#x27;verification&#x27;)
- | | | └─ Audio Scrubbing Player (Auto-loops 5s flag snippet)
- | | | └─ Progress Completion Bar (X of Y resolved)
- | | | └─ Flagged Items List (VerificationItemRow)
- | | | | └─ Confirm Text Action
- | | | | └─ Apply Correction Action
- | | | | └─ Mark False Positive Action
- | | | └─ Add Manual Verification Point Form
- | | | └─ Finalise & Approve Transcript Action
- | | |
- | | └─ Sub-View 5.3: Verified Transcript Viewer (&#x27;verified\_transcript&#x27;)
- | | | └─ Clean Approved Transcript Text
- | | | └─ Verification Audit Log (Who verified, changes made, timestamp)
- | | | └─ Proceed to Reporting Button
- | | |
- | | └─ Sub-View 5.4: ReportingView (&#x27;reporting&#x27;)
- | | | └─ Active Standard Banner & Modal Trigger (ReportingStandardsModal)
- | | | └─ Generate Dual Reports Trigger (Gemini API)
- | | | └─ Side-by-Side Dual Reporter Cards (ReportCard: Reporter A vs Reporter B)
- | | | └─ Proceed to Editing Stage Button
- | | |
- | | └─ Sub-View 5.5: EditingView (&#x27;editing&#x27;)
- | | | └─ Active Standard Banner & Modal Trigger (EditorStandardsModal)
- | | | └─ AI Synthesis Generator Button
- | | | └─ Source Reference Flyout Drawer (SourceReferenceDrawer)
- | | | | └─ Tab 1: Reporter A Draft
- | | | | └─ Tab 2: Reporter B Draft
- | | | | └─ Tab 3: Verified Full Transcript
- | | | └─ Markdown Editorial Text Editor
- | | | └─ Word Count & Save Draft Button
- | | | └─ Approve & Finalise Editorial Draft Button
- | | |
- | | └─ Sub-View 5.6: ProofreadingView (&#x27;proofreading&#x27;)
- | | | └─ Active Standard Banner & Modal Trigger (ProofreadingStandardsModal)
- | | | └─ Run Proofreading Inspection Trigger
- | | | └─ Dual-Pane Proofreading Inspection:
  - | | | | └─ Left Pane: Marked-up text with green insertions and red strikethroughs
  - | | | | └─ Right Pane: Proposed Changes List (ProofreadingChangesList)

- | | | | — Accept / Reject Individual Change Buttons
- | | | | — Accept All High-Confidence Changes Button
- | | | | — Finalise Proofreading & Proceed Button
- | | | |
- | | | — Sub-View 5.7: FinalReportView (&#x27;final\_report&#x27;)
- | | | | — Official Document Layout Preview
- | | | | — Inline Edit Capability
- | | | | — Download Word Document (.docx) Button
- | | | | — Copy Formatted Text to Clipboard Button
- | | | | — Print / Save as PDF Button
- | | | |
- | | — [View 6] Transcribe Pipeline (Route: &#x27;transcribe&#x27;)
- | | | — Left Column:
- | | | | — RecordedFileUploader (Drag-drop area, file chip, provider radio)
- | | | | — TranscriptionProgress (Upload progress bar, status polling, error retry)
- | | | — Right Column: TranscriptsHistoryList (Past file transcript jobs)
- | | | — Full-Screen Result: RawTranscriptViewer (Rendered when transcript active)
- | | | |
- | | — [View 7] YouTubeSessionView (Route: &#x27;youtube&#x27;)
- | | | — Mode Tabs: Live Tab Audio Capture vs Archived YouTube URL
- | | | — Live Tab Capture Card (Chrome tab audio instructions + trigger)
- | | | — URL Ingestion Form (URL input, video preview metadata, download & transcribe)
- | | | — Recent YouTube Sessions List
- | | | |
- | | — [View 8] SettingsView (Route: &#x27;settings&#x27;)
- | | | — Section 1: ProgrammesSettingsSection
- | | | | — Add Programme Form
- | | | | — Programmes Accordion List (Rename, Archive)
- | | | | — Child Sections List (Add section, Rename, Archive)
- | | | — Section 2: AI Standards Tabs (Reporting, Editing, Proofreading)
- | | | | — Active Version Indicator & Save Button
- | | | | — Textarea Prompt Editors (Guidelines, Reporter A/B, Glossaries, Examples)
- | | | | — Version History Modals (ReportingStandardsModal, EditorStandardsModal, ProofreadingStandardsModal)
- | | | |
- | | | — Section 3: Transcription Providers Status Grid (Azure Speech, Faster-Whisper, Google Cloud, Server status)

## 5. CURRENT DESIGN SYSTEM

This section specifies the visual and ergonomic rules currently declared in `frontend/src/App.css` (10,429 lines, 193 KB).

### 5.1 Color Palette & Tokens

The application relies on CSS custom properties declared at `:root` in `frontend/src/App.css`:

| CSS Token         | Hex Value | Name / Purpose | Real-World Usage in App                                                    |
|-------------------|-----------|----------------|----------------------------------------------------------------------------|
| `--color-primary` | `#163e73` | Deep Navy Blue | Primary buttons, active sidebar links, brand mark dot, active stepper node |

| CSS Token                          | Hex Value            | Name / Purpose     | Real-World Usage in App                                                       |
|------------------------------------|----------------------|--------------------|-------------------------------------------------------------------------------|
| <code>--color-primary-dark</code>  | <code>#0f2947</code> | Midnight Navy      | Primary button <code>:hover</code> , dark headings, top-level text contrast   |
| <code>--color-primary-light</code> | <code>#eaf3fc</code> | Ice Blue Tint      | Active sidebar item background, table row highlight, info banners             |
| <code>--color-bg</code>            | <code>#F5F6F8</code> | Light Cool Grey    | Master viewport canvas background, body background behind cards               |
| <code>--color-card-bg</code>       | <code>#ffffff</code> | Pure White         | Surface cards, modals, dropdown menus, table bodies, input fields             |
| <code>--color-text-main</code>     | <code>#0f2947</code> | High-Contrast Navy | Page titles, card headers, table headers, stat numbers                        |
| <code>--color-text-body</code>     | <code>#334155</code> | Slate Grey         | Body copy, transcript text, form descriptions, table cell text                |
| <code>--color-text-muted</code>    | <code>#64748b</code> | Muted Blue-Slate   | Timestamps, secondary subtitles, placeholder text, inactive steps             |
| <code>--color-border</code>        | <code>#e2e8f0</code> | Border Grey        | 1px card outlines, divider lines, table borders, inactive button borders      |
| <code>--color-success</code>       | <code>#15803d</code> | Forest Green       | Completed status badges, audio level meter safe zone, confirmed flags         |
| <code>--color-warning</code>       | <code>#d97706</code> | Deep Amber         | Interrupted session warnings, needs verification pill, audio level peak       |
| <code>--color-danger</code>        | <code>#dc2626</code> | Crimson Red        | Recording pulse dot, stop recording button, delete action, audio clip warning |
| <code>--color-danger-light</code>  | <code>#fef2f2</code> | Soft Pink Tint     | ErrorBanner background, discarded verification item background                |

Brand Legacy Comment: While early CSS comments reference `#338FE0` and `#E74C3C`, the running system standardizes on the deeper corporate `#163e73` navy palette.

## 5.2 Typography System

- **Primary Sans-Serif:** `Inter`, `-apple-system`, `BlinkMacSystemFont`, `"Segoe UI"`, `Roboto`, `Oxygen`, `Ubuntu`, `Cantarell`, `sans-serif`;
- Body line height: `1.55`
- Font smoothing: `-webkit-font-smoothing: antialiased`; `-moz-osx-font-smoothing: grayscale`;
- **Monospace Font:** `ui-monospace`, `SFMono-Regular`, `"SF Mono"`, `Menlo`, `Consolas`, `"Liberation Mono"`, `monospace`;
- Used for raw transcript debug dumps, JSON inspection, and provider technical details.
- **Tabular Numerals** (`font-variant-numeric: tabular-nums`; `font-feature-settings: "tnum" 1`);
- Universally enforced across all numeric data: `.cell-date`, `.cell-duration`, `.time-stamp`, `.session-id-badge`, `.sessions-count-pill`, `.stat-card-stitch`, and live timers to prevent visual jitter when numbers change.
- **Typographic Hierarchy:**
  - View / Page Titles (`h1` / `.screen-title`): `1.35rem` to `1.5rem`, Weight `700`, Letter-spacing `-0.02em`, Color: `#0f2947`.
  - Section Headings (`h2` / `.section-title`): `1.15rem` to `1.25rem`, Weight `600`, Color: `#0f2947`.
  - Card Titles (`h3` / `.card-title`): `0.95rem` to `1.05rem`, Weight `600`.
  - Body Copy (`p`, `span`): `0.875rem` (14px), Weight `400`, Color: `#334155`.
  - Metadata / Small Captions: `0.75rem` (12px), Weight `500`, Color: `#64748b`.
  - Big Metrics (`.stat-number`): `1.85rem` to `2.25rem`, Weight `700`, Line-height `1.1`.

## 5.3 Spacing, Borders & Shadows

- **Border Radii:**
  - `--radius-sm: 6px` (Used for buttons, input fields, badges, tags, pills).
  - `--radius-md: 10px` (Used for surface cards, dialog modals, drawer panels, dropzones).
- **Elevation & Shadows:**
  - `--shadow-sm: 0 1px 3px rgba(0, 0, 0, 0.05)` (Subtle separation for header, inactive cards).
  - `--shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.07)` (Hover cards, active dropdowns, modal dialogs).
  - Floating Controller Shadow: `0 8px 24px rgba(0, 0, 0, 0.15)` (Floating recorder elevation).
- **Borders:**
  - Standard border: `1px solid var(--color-border) (#e2e8f0)`.
  - Active / Focus outline: `2px solid var(--color-primary)` with `2px` offset.

## 5.4 Icon Strategy

- **Current Implementation: No dedicated SVG icon library is installed** (no Lucide, Heroicons, or Feather).
- **Approach:**
- **Unicode Emojis / Symbols:** Used extensively in sidebar (🏠, 📁, 📄, 📑, 📊, 📈), buttons (🔍, 🗑️, 🔄, 📄, 📁, 📑, 📊, 📈), and status messages.
- **Raw Inline SVGs:** Used in specific interactive widgets: audio play/pause buttons, chevron dropdown arrows, and close 🗑️ buttons.
- **UX Impact:** Inconsistent rendering across Windows, macOS, Android, and iOS (e.g. Windows Segoe UI Emoji renders flat outlines or mismatched colors compared to Apple Color Emoji).

## 5.5 Button Variants & Interactive States

- **Primary (`.btn--primary`):** Background `#163e73`, Text `#ffffff`, Border none, Border-radius `6px`, Padding `0.55rem 1.15rem`, Font weight `600`. Hover: `#0f2947`.
- **Secondary (`.btn--secondary`):** Background `#ffffff`, Text `#0f2947`, Border `1px solid #e2e8f0`, Hover: Background `#f8fafc`.
- **Danger (`.btn--danger`):** Background `#dc2626`, Text `#ffffff`. Hover: `#b91c1c`. Used for Stop Recording, Delete Session.
- **Subtle / Ghost (`.btn--subtle`):** Background transparent, Text `#64748b`, Hover: Background `#eaf3fc`, Text `#0f2947`.
- **Sizes:** `.btn--sm` (padding `0.35rem 0.75rem`, 12px font), `.btn--lg` (padding `0.75rem 1.5rem`, 16px font).
- **Disabled State:** Opacity `0.5`, `cursor: not-allowed`, pointer events blocked.

## 5.6 Form Controls & Input Styling

- **Text Inputs & Selects:**
  - Background `#ffffff`, Border `1px solid #e2e8f0`, Border-radius `6px`, Height `38px` (padding `0.45rem 0.75rem`), Font size `0.875rem`.
  - Focus: Border color `#163e73`, Box-shadow `0 0 3px rgba(22, 62, 115, 0.12)`.
- **Textareas:**
  - Min-height `100px`, Line-height `1.5`, Font-family `var(--font-sans)`.
- **Radio Groups & Checkboxes:**
  - Native browser inputs styled with accent color `#163e73`.

## 5.7 Cards, Tables, Modals & Drawers

- **Cards (`.card`, `.stat-card-stitch`):**

- Background ``#ffffff``, Border ``1px solid #e2e8f0``, Border-radius ``10px``, Padding ``1.25rem``, Box-shadow ``var(--shadow-sm)``.
- **Tables (`.sessions-table``):**
  - Width ``100%``, Border-collapse ``separate``, Border-spacing ``0``.
  - Header ``th``: Background ``#f8fafc``, Font size ``0.75rem``, Text-transform ``uppercase``, Letter spacing ``0.05em``, Color ``#64748b``, Border-bottom ``1px solid #e2e8f0``, Padding ``0.75rem 1rem``.
  - Rows ``tr``: Hover background ``#f8fafc``. Padding ``0.75rem 1rem``.
- **Modals (`.modal-backdrop``, `.modal-dialog``):**
  - Backdrop: ``rgba(15, 41, 71, 0.45)`` with ``backdrop-filter: blur(3px)``.
  - Dialog: Centered, Max-width ``680px`` (or ``900px`` for standards), Max-height ``90vh``, Overflow-y ``auto``, Border-radius ``12px``, Box-shadow ``0 20px 25px -5px rgba(0, 0, 0, 0.2)``.
- **Flyout Drawer (`.flyout-drawer``):**
  - Fixed to right edge, Width ``440px`` (Max ``90vw``), Height ``100vh``, Z-index ``1000``, Box-shadow ``-5px 0 25px rgba(0, 0, 0, 0.15)``.

## 5.8 Badges, Pills & Status Indicators

- **Status Badges (`.status-pill``):**
  - Completed: Green background ``#dcfce7``, Text ``#15803d``.
  - Recording: Red background ``#fee2e2``, Text ``#dc2626``, Pulsing red dot.
  - Needs Verification: Amber background ``#fef3c7``, Text ``#d97706``.
  - Interrupted: Deep amber background ``#ffedd5``, Text ``#c2410c``.
  - Not Started / Pending: Grey background ``#f1f5f9``, Text ``#64748b``.
- **Flag Count Badge (`.flag-badge``):**
  - Circular or rounded pill, Background ``#fee2e2``, Text ``#dc2626``, Font weight ``700``.

## 6. COMPONENT INVENTORY

This section catalogs all reusable frontend components, their properties, internal states, visual variants, and architectural overlap.

### Component 1: ``AppShell``

- **File:** ``frontend/src/components/common/AppShell.jsx``
- **Purpose:** The master layout wrapper containing the persistent left sidebar navigation and dynamic top header.
- **Props Received:**
  - ``activeView``: String (``'dashboard' | 'sessions' | 'new_live' | 'transcribe' | 'settings' | 'youtube' | 'live_recording'``).
  - ``isLiveRecordingActive``: Boolean.
  - ``onNavigate``: Function ``(viewName) => void``.
  - ``screenTitle``: String.
  - ``onBack``: Function ``() => void`` or ``null``.
  - ``onStartLiveSession``: Function ``() => void``.
  - ``children``: ``ReactNode``.
- **Internal State:** ``mobileMenuOpen`` (Boolean, for responsive drawer).
- **Visual Variants:** Standard Desktop layout vs Mobile drawer layout with backdrop.
- **Reusability & Duplication Notes:** Central layout used on 100% of screens. Cleanly centralized.

## Component 2: `LifecycleStepper`

- **File:** `frontend/src/components/common/LifecycleStepper.jsx`
- **Purpose:** Renders the 8-stage horizontal progress pipeline across church session processing.
- **Props Received:**
  - `currentStage`: String (`'audio'` | `'raw\_transcript'` | `'verification'` | `'verified\_transcript'` | `'reporting'` | `'editing'` | `'proofreading'` | `'final\_report'`).
  - `stageStatuses`: Object mapping stage keys to status strings (`'not\_started'` | `'in\_progress'` | `'complete'` | `'ready'` | `'locked'`).
  - `onSelectStage`: Function `(stageKey) => void` (optional stage jump).
- **Internal State:** None (Pure presentational).
- **Visual Variants:** Node states: Completed (Green check), Active (Blue circle with pulse), Locked (Grey padlock), Warning (Amber flag).
- **Reusability & Duplication Notes:** Rendered at top of `SessionDetailView`. Reused across Session Workspace and stage sub-views.

## Component 3: `ErrorBanner`

- **File:** `frontend/src/components/ErrorBanner.jsx`
- **Purpose:** Standardized dismissible error notification bar.
- **Props Received:**
  - `error`: String or Error Object.
  - `onDismiss`: Function `() => void`.
- **Internal State:** None.
- **Visual Variants:** Soft red background (`#fef2f2`) with dark red text and close `✕` button.
- **Reusability & Duplication Notes:** Reused in `App.jsx`, `RecordedFileUploader.jsx`, `VerificationWorkflow.jsx`, and `ReportingView.jsx`.

## Component 4: `ReportCard`

- **File:** `frontend/src/components/reporting/ReportCard.jsx`
- **Purpose:** Card container displaying an individual AI reporter summary (Reporter A or Reporter B).
- **Props Received:**
  - `reporterRole`: `reporter\_a` | `reporter\_b`.
  - `report`: Object (Report record with text, status, model\_name, created\_at, key\_points, scriptures).
  - `onRegenerate`: Function.
  - `onCopy`: Function.
  - `isLoading`: Boolean.
- **Internal State:** `copied` (Boolean for copy confirmation tooltip).
- **Visual Variants:** Reporter A theme (Navy accent border) vs Reporter B theme (Slate accent border).
- **Reusability & Duplication Notes:** Cleanly reused twice side-by-side in `ReportingView.jsx`.

## Component 5: `ReportingStandardsModal`

- **File:** `frontend/src/components/reporting/ReportingStandardsModal.jsx`
- **Purpose:** Modal dialog displaying reporting prompt guidelines and version history.
- **Props Received:** `isOpen`, `onClose`, `standards`, `onSaveStandard`, `activeVersion`.
- **Internal State:** Selected version for rollback, active tab.

- **Reusability & Duplication Notes:** Significant structural overlap with ``EditorStandardsModal`` and ``ProofreadingStandardsModal``. All three modals share 80% identical dialog layout and version history logic.

## Component 6: ``EditorStandardsModal``

- **File:** ``frontend/src/components/editing/EditorStandardsModal.jsx``
- **Purpose:** Modal dialog for editing compilation guidance, glossaries, and prompt versions.
- **Props Received:** ``isOpen``, ``onClose``, ``standards``, ``onSaveStandard``.
- **Reusability & Duplication Notes:** Duplicate pattern of ``ReportingStandardsModal``.

## Component 7: ``SourceReferenceDrawer``

- **File:** ``frontend/src/components/editing/SourceReferenceDrawer.jsx``
- **Purpose:** Slide-out inspection drawer allowing editors to consult Reporter A, Reporter B, or full transcript while editing.
- **Props Received:**
  - ``isOpen``: Boolean.
  - ``onClose``: Function.
  - ``reporterADraft``: Object.
  - ``reporterBDraft``: Object.
  - ``verifiedTranscript``: String / Array.
- **Internal State:** ``activeTab`` ('reporter\_a' | 'reporter\_b' | 'transcript'), ``searchQuery`` (filtering transcript text).
- **Visual Variants:** Slide-in animation from right edge with tab pills.
- **Reusability & Duplication Notes:** High UX value component; currently used exclusively in ``EditingView.jsx``.

## Component 8: ``ProofreadingStandardsModal``

- **File:** ``frontend/src/components/proofreading/ProofreadingStandardsModal.jsx``
- **Purpose:** Modal for proofreading capitalization, terminology, and scripture formatting rules.
- **Props Received:** ``isOpen``, ``onClose``, ``standards``, ``onSaveStandard``.
- **Reusability & Duplication Notes:** Triplicate of the standards modal pattern.

## Component 9: ``ProofreadingChangesList``

- **File:** ``frontend/src/components/proofreading/ProofreadingChangesList.jsx``
- **Purpose:** Renders the categorized list of proposed AI proofreading changes with accept/reject buttons.
- **Props Received:**
  - ``changes``: Array of change objects `{ id, category, original, replacement, reason, accepted }`.
  - ``onAccept``: Function ``(changelid) => void``.
  - ``onReject``: Function ``(changelid) => void``.
  - ``onAcceptAll``: Function ``() => void``.
- **Internal State:** Category filter ('all' | 'grammar' | 'divine\_pronoun' | 'scripture' | 'terminology').
- **Visual Variants:** Pending row (Amber highlight) vs Accepted row (Green check) vs Rejected row (Strikethrough).
- **Reusability & Duplication Notes:** Specialized component for Stage 6.

## Component 10: ``AudioLevelMeter``

- **File:** ``frontend/src/components/AudioLevelMeter.jsx``
- **Purpose:** Real-time visual VU meter visualizing microphone input levels in dBFS.

- **Props Received:**
  - ``level``: Number (0.0 to 1.0 or -60dB to 0dB).
  - ``hasSignal``: Boolean.
  - ``orientation``: `"horizontal"` | `"vertical"`.
- **Internal State:** Peak hold decay timer.
- **Visual Variants:** Green segment (safe: 0-70%), Amber segment (warm: 70-85%), Red segment (clipping: 85-100%).
- **Reusability & Duplication Notes:** Reused in ``NewLiveSessionView``, ``LiveRecordingView``, and ``FloatingRecordingController``.

## Component 11: ``AudioSourceSelector``

- **File:** ``frontend/src/components/AudioSourceSelector.jsx``
- **Purpose:** Dropdown picker enumerating connected hardware audio inputs via ``navigator.mediaDevices.enumerateDevices()``.
- **Props Received:**
  - ``selectedDeviceId``: String.
  - ``onDeviceSelect``: Function ``(deviceId) => void``.
  - ``disabled``: Boolean.
- **Internal State:** ``devices`` (Array of `MediaDeviceInfo`), ``permissionGranted`` (Boolean).
- **Reusability & Duplication Notes:** Reused in ``NewLiveSessionView`` and ``SettingsView``.

## Component 12: ``CompletedRecordingPlayer``

- **File:** ``frontend/src/components/CompletedRecordingPlayer.jsx``
- **Purpose:** Custom audio playback toolbar with play/pause, scrub bar, time display, playback rate (``0.75x``, ``1.0x``, ``1.25x``, ``1.5x``), and loop snippet controls.
- **Props Received:**
  - ``audioUrl``: String.
  - ``currentTime``: Number.
  - ``onTimeUpdate``: Function.
  - ``mediaElementRef``: React ref.
- **Internal State:** ``isPlaying``, ``playbackRate``, ``volume``.
- **Reusability & Duplication Notes:** Reused in ``VerificationWorkflow.jsx`` and ``RawTranscriptViewer.jsx``.

## Component 13: ``VerificationItemRow``

- **File:** ``frontend/src/components/verification/VerificationItemRow.jsx``
- **Purpose:** Renders an individual flagged transcript item requiring verification.
- **Props Received:**
  - ``item``: Object (flagged segment).
  - ``isSelected``: Boolean.
  - ``onSelect``: Function.
  - ``onConfirm``: Function.
  - ``onCorrect``: Function ``(newText) => void``.
  - ``onDiscard``: Function.
- **Internal State:** ``isEditing``, ``inputText``.
- **Visual Variants:** Pending state, Active selected state (blue border), Resolved state (green check).
- **Reusability & Duplication Notes:** Clean single-purpose list item component.

# 7. DATA & DOMAIN MODEL

The application stores all data persistently in SQLite at `storage/app.db`. There are 12 relational database tables defined in `backend/app/database/models.py`.

## 7.1 Entity Explanations & Relational Schema



### 1. `sessions` (Master Record)

- **Purpose:** The root entity representing a single recorded church service, sermon, or ingested audio file.
- **Key Fields:**
  - `session\_id` (TEXT, PK): Unique identifier (e.g. `sess\_20260927\_101500`).
  - `title` (TEXT): Display title of the session / sermon.
  - `date\_created` (TEXT, ISO-8601): Creation timestamp.
  - `status` (TEXT): Lifecycle status (`recording`, `completed`, `interrupted`, `verified`, `reporting`, `editing`, `proofreading`, `finalized`).
  - `duration\_seconds` (REAL): Total audio duration.
  - `audio\_filename`, `audio\_file\_path`, `audio\_file\_size` (TEXT, INTEGER): Audio file storage references.
  - `transcript\_id`, `raw\_text` (TEXT): Consolidated raw transcript string.
  - `provider\_name` (TEXT): AI provider used (`azure\_speech` or `faster\_whisper`).
  - `segment\_count`, `flag\_count` (INTEGER): Number of chunks and detected flags.
  - `is\_interrupted`, `recovery\_notes` (INTEGER, TEXT): Operational crash recovery markers.
  - `metadata\_json` (TEXT): JSON containing minister, programme, section, and sermon notes.
  - Lifecycle tracking columns: `verification\_status`, `reporting\_status`, `editing\_status`, `proofreading\_status`, `final\_report\_status`.

### 2. `session\_segments` (Raw STT Chunks)

- **Purpose:** Timestamped speech chunks generated during live or file transcription.
- **Key Fields:** `segment\_id` (PK), `session\_id` (FK -> sessions), `segment\_index` (INT), `start\_time` (REAL), `end\_time` (REAL), `text` (TEXT), `confidence` (REAL), `is\_low\_confidence` (BOOL), `flags\_json` (TEXT), `words\_json` (TEXT - word-level timestamps).

### 3. `verification\_items` (Human-in-the-Loop Flags)

- **Purpose:** Flagged segments requiring human verification due to low confidence, unfamiliar biblical names, or Scripture citations.

- **Key Fields:** `item\_id` (PK), `session\_id` (FK -> sessions), `segment\_index` (INT), `original\_text` (TEXT), `verified\_text` (TEXT), `start\_time` (REAL), `end\_time` (REAL), `action` (TEXT: `pending` | `confirmed` | `corrected` | `discarded`), `flag\_reasons` (TEXT), `verified\_at` (TEXT).

#### 4. `reporting\_standards` (Prompt Engineering Templates)

- **Purpose:** Versioned instructions governing how Gemini produces Stage 4 Reporter A and Reporter B summaries.
- **Key Fields:** `id` (PK), `version` (INT, UNIQUE), `version\_label` (TEXT), `is\_active` (BOOL), `general\_guidelines` (TEXT), `reporter\_a\_instructions` (TEXT), `reporter\_b\_instructions` (TEXT), `terminology` (TEXT), `examples` (TEXT), `created\_at` (TEXT).

#### 5. `reports` (Dual AI Summaries)

- **Purpose:** Individual summaries generated by Reporter A (thematic outline) or Reporter B (narrative details).
- **Key Fields:** `report\_id` (PK), `session\_id` (FK -> sessions), `reporter\_role` (TEXT: `reporter\_a` | `reporter\_b`), `standard\_version` (INT), `status` (TEXT: `generating` | `ready` | `failed`), `report\_title` (TEXT), `report\_text` (TEXT), `key\_points\_json` (TEXT), `scriptures\_json` (TEXT), `model\_name` (TEXT: `gemini-2.5-flash`), `is\_active` (BOOL).

#### 6. `editor\_standards` (Editorial Guidelines)

- **Purpose:** Versioned instructions for synthesizing dual reports into a unified master editorial draft.
- **Key Fields:** `id` (PK), `version` (INT), `is\_active` (BOOL), `general\_guidelines` (TEXT), `compilation\_guidance` (TEXT), `terminology` (TEXT), `approved\_examples` (TEXT).

#### 7. `edited\_reports` (Chief Editor Revisions)

- **Purpose:** Master editorial drafts produced by AI synthesis or edited directly by the human Chief Editor.
- **Key Fields:** `revision\_id` (PK), `session\_id` (FK -> sessions), `revision\_number` (INT), `revision\_source` (TEXT: `ai\_generated` | `human\_edited` | `ai\_regenerated`), `report\_title` (TEXT), `report\_text` (TEXT), `review\_notes\_json` (TEXT), `is\_active` (BOOL).

#### 8. `proofreading\_standards` (Style & Orthography Rules)

- **Purpose:** Versioned rules for King James Scripture references, divine pronoun capitalization, and church nomenclature.
- **Key Fields:** `id` (PK), `version` (INT), `is\_active` (BOOL), `guidelines` (TEXT), `terminology` (TEXT), `formatting\_rules` (TEXT).

#### 9. `proofread\_reports` (Proofreading Revisions)

- **Purpose:** Proofread drafts containing inspected text, tracked diffs, and acceptance records.
- **Key Fields:** `revision\_id` (PK), `session\_id` (FK -> sessions), `revision\_number` (INT), `revision\_source` (TEXT: `ai\_proofread` | `human\_reviewed`), `proofread\_text` (TEXT), `changes\_json` (TEXT), `is\_active` (BOOL), `is\_accepted` (BOOL).

#### 10. `final\_reports` (Publication-Ready Deliverables)

- **Purpose:** The finalized sermon publication record with generated Microsoft Word `.docx` file attachments.
- **Key Fields:** `id` (PK), `session\_id` (FK -> sessions), `revision\_number` (INT), `report\_title` (TEXT), `report\_text` (TEXT), `minister` (TEXT), `programme` (TEXT), `service\_date` (TEXT), `docx\_filename` (TEXT), `docx\_file\_size` (INT), `is\_active` (BOOL).

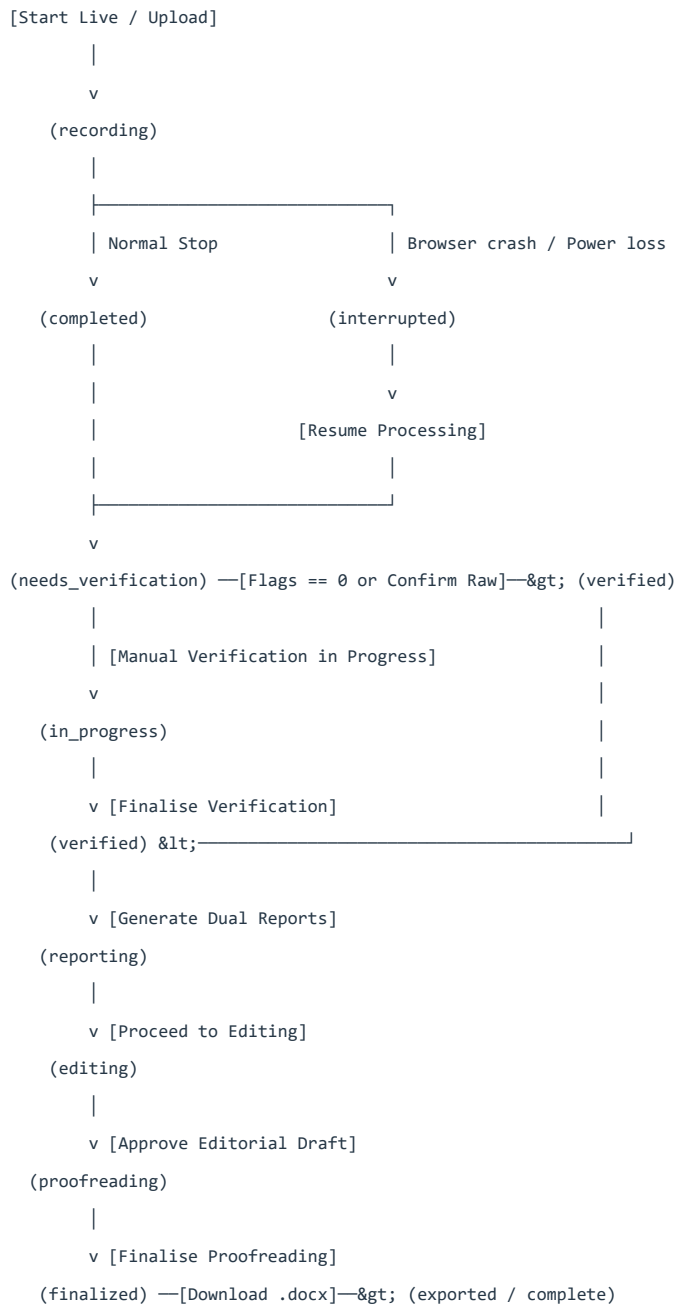
#### 11. `programmes` (Church Programme Taxonomy)

- **Purpose:** Top-level church events (e.g. "Sunday Worship Service", "Monday Bible Study", "Dec Retreat").
- **Key Fields:** `id` (PK), `name` (TEXT), `is\_archived` (BOOL), `sort\_order` (INT).

#### 12. `programme\_sessions` (Child Sections Taxonomy)

- **Purpose:** Specific services within a programme (e.g. "Morning Message", "Faith Clinic", "Youth Service").
- **Key Fields:** `id` (PK), `programme\_id` (FK -> programmes), `name` (TEXT), `is\_archived` (BOOL), `sort\_order` (INT).

## 7.2 Session Lifecycle State Machine



## 8. ROLES & PERMISSIONS

This section clarifies the operational reality of user access versus the conceptual roles described in product planning documentation.

### 8.1 The Implementation Reality vs. Planned Roles

- **Current Reality:**
- **There is NO login page, authentication gate, session token, or role-based access control (RBAC) in the codebase.**
- The application currently operates as an **internal single-operator desktop tool**.

- Any person who opens the browser tab has full, unrestricted read/write access to every screen, table, prompt template, database record, and configuration setting.
- **Planned Conceptual Roles (from `PROJECT\_SCOPE.md`):**
  - Audio Technician / Capture Operator: Responsible for live microphone calibration, YouTube ingestion, file uploads, and monitoring audio signal dBFS.
  - Verification Specialist: Responsible for audio scrubbing, resolving flagged phonemes / biblical terms, and approving verified transcripts.
  - Reporter (AI Operator): Responsible for triggering dual reporting summaries and comparing Reporter A and Reporter B drafts.
  - Chief Editor: Responsible for editorial synthesis, using the source flyout drawer, and approving publication drafts.
  - Proofreader: Responsible for style-guide adherence, Scripture citation cross-referencing, and divine pronoun formatting.
  - Administrator / Lead Pastor: Responsible for church programmes taxonomy and modifying underlying AI prompt standards.
  - Church Media Consumer: Read-only access to final reports, sermon summaries, and Word `.docx` downloads.

## 8.2 State-Driven Feature Locking (Data Guardrails)

While there is no user-based authentication, the application strictly enforces **pipeline dependency locking** in the UI:

- An operator **cannot** generate dual reports until verification is finalized (or confirmed as verified).
- An editor **cannot** open the synthesis workspace until at least one report exists.
- A proofreader **cannot** run style inspections until an editorial draft is approved.
- The final `.docx` download is locked until proofreading is approved.
- **Lock Indication in UI:** Upstream stage tiles in `SessionDetailView` display padlock icons (`🔒 Locked`), muted grey backgrounds, and disabled action buttons until previous requirements are satisfied.

# 9. FORMS INVENTORY

This section catalogs all 14 interactive forms and input surfaces across the application.

## Form 1: New Live Session Setup Form

- **Location:** `NewLiveSessionView.jsx`
- **Form Inputs:**
  - Church Programme: HTML ``, required. Populated dynamically from `/api/programmes`.
  - Programme Session / Section: HTML ``, required. Populates based on selected programme.
  - Session / Sermon Title: HTML `  - Minister / Speaker: HTML `` with text override, options: "Pastor W.F. Kumuyi", "Guest Speaker", or custom text.
  - Message Title: HTML `  - Audio Input Device: Custom `` (`AudioSourceSelector.jsx`), enumerating system microphones.
  - Transcription Provider: Radio group: `Azure Speech (Recommended)` vs `Faster-Whisper (Offline)`.
- **Validation & Error Handling:**
  - Blocks submission if title or programme is blank (red border + inline message).
  - Warns if no microphone device is selected or audio signal is muted (dBFS below -60dB).
- **Submission Behavior:** Calls `handleStartRecording()`, switches view to `live\_recording`, initializes Web Audio stream and SQLite session record.

## Form 2: Inline Session Title Editor

- **Location:** `SessionDetailView.jsx` (Header bar)
- **Form Inputs:** HTML `- **Validation & Error Handling:** Trims whitespace; disables `Save` button if empty.
- **Submission Behavior:** Calls `onUpdateTitle(sessionId, newTitle)`. Sends `PATCH /api/sessions/{id}`.

## Form 3: Flagged Verification Item Correction Form

- **Location:** `VerificationItemRow.jsx`
- **Form Inputs:**
  - Corrected Text: HTML `  - Correction Note: Optional note input for theological context.
- **Validation & Error Handling:** Allows enter key to submit; empty string warns before discarding.
- **Submission Behavior:** Sends `PUT /api/verification/items/{id}` with action `corrected` and advances audio player.

## Form 4: Manual Verification Point Form

- **Location:** `VerificationWorkflow.jsx`
- **Form Inputs:**
  - Timestamp: Seconds input or current audio scrubber position.
  - Word / Phrase to Flag: Text input.
  - Reason for Flag: Dropdown (`Scripture Reference`, `Proper Name`, `Theological Term`, `Unclear Audio`).
- **Validation & Error Handling:** Validates timestamp falls within audio duration.
- **Submission Behavior:** Inserts new row into `verification\_items` table.

## Form 5: Reporting Standards Editor

- **Location:** `SettingsView.jsx` (Reporting Tab) & `ReportingStandardsModal.jsx`
- **Form Inputs:**
  - General Guidelines: Multi-line `  - Reporter A Instructions: Multi-line `  - Reporter B Instructions: Multi-line `  - Church Terminology & Glossary: Multi-line `  - Approved Few-Shot Examples: Multi-line `  - Change Note: Single-line `
- **Validation & Error Handling:** Requires non-empty instructions; displays confirmation modal before overwriting active prompt.
- **Submission Behavior:** Sends `POST /api/standards/reporting`, increments version counter, sets `is\_active = 1`.

## Form 6: Editor Standards Editor

- **Location:** `SettingsView.jsx` (Editing Tab) & `EditorStandardsModal.jsx`
- **Form Inputs:** General Guidelines, Compilation Guidance, Church Terminology, Approved Examples, Change Note.
- **Validation & Error Handling:** Required fields check.
- **Submission Behavior:** Sends `POST /api/standards/editing`.

## Form 7: Proofreading Standards Editor

- **Location:** `SettingsView.jsx` (Proofreading Tab) & `ProofreadingStandardsModal.jsx`
- **Form Inputs:** Proofreading Guidelines, Church Glossary, Formatting Rules (Divine Pronouns, Scripture Citations), Change Note.
- **Validation & Error Handling:** Required fields check.
- **Submission Behavior:** Sends `POST /api/standards/proofreading`.

## Form 8: Editorial Workbench Textarea

- **Location:** `EditingView.jsx`
- **Form Inputs:** Full-height markdown ``<textarea>`` displaying master synthesized sermon draft.
- **Validation & Error Handling:** Word count monitor updates live; warns if trying to exit with unsaved changes.
- **Submission Behavior:** `Save Draft` saves revision without closing; `Approve Editorial Draft` marks editing `complete`.

## Form 9: Final Report Text Preview Editor

- **Location:** `FinalReportView.jsx`
- **Form Inputs:** Content-editable document preview / textarea for making final headline, author, or typo tweaks.
- **Validation & Error Handling:** Auto-saves before triggering `.docx` file generation.
- **Submission Behavior:** Generates Microsoft Word document via `POST /api/final-report/{id}/generate-docx`.

## Form 10: Add / Rename Church Programme Form

- **Location:** `ProgrammesSettingsSection.jsx`
- **Form Inputs:**
  - Programme Name: HTML ``<input type="text">``, placeholder: "e.g. December National Retreat".
- **Validation & Error Handling:** Rejects duplicate names or blank strings with inline red alert.
- **Submission Behavior:** Sends `POST /api/programmes` or `PATCH /api/programmes/{id}`.

## Form 11: Add / Rename Child Section Form

- **Location:** `ProgrammesSettingsSection.jsx`
- **Form Inputs:**
  - Section Name: HTML ``<input type="text">``, placeholder: "e.g. Faith Clinic".
- **Validation & Error Handling:** Validates non-empty string.
- **Submission Behavior:** Sends `POST /api/programmes/{programme\_id}/sessions`.

## Form 12: YouTube Ingestion URL Form

- **Location:** `YouTubeSessionView.jsx`
- **Form Inputs:**
  - YouTube Video URL: HTML ``<input type="url">``, placeholder: "https://www.youtube.com/watch?v=..."
  - Programme Selector: Dropdown.
  - Section Selector: Dropdown.
- **Validation & Error Handling:** Validates regex for valid YouTube domain (`youtube.com` or `youtu.be`).
- **Submission Behavior:** Triggers `POST /api/youtube/fetch-info` followed by background `yt-dlp` download.

Form 13: Recorded File Upload & Provider Selector

- **Location:** `RecordedFileUploader.jsx`
- **Form Inputs:**
  - Drag-and-Drop Dropzone: Hidden `<input type="file" accept="audio/,video/">`.
  - Provider Radio Group: `Azure Speech` vs `Faster-Whisper`.
- **Validation & Error Handling:** Checks file size against 500MB ceiling and MIME format.
- **Submission Behavior:** Multi-part form data upload to `/api/transcribe/upload``.

Form 14: Sessions Directory Filter & Search Bar

- **Location:** `SessionHistoryList.jsx`
- **Form Inputs:**
  - Search Query: `<input type="search">`, placeholder: `""Search by title or minister..."`.
  - Programme Filter: `<select>` dropdown.
  - Status Tabs: Buttons (``All``, ``Needs Verification``, ``Completed``, ``Interrupted``).
- **Validation & Error Handling:** Real-time client-side and server-side debounced filtering.
- **Submission Behavior:** Modifies query params or re-fetches ``/api/sessions``.

10. CURRENT RESPONSIVE BEHAVIOUR

This section audits how the application responds across standard viewport breakpoints based on direct headless browser testing and ``App.css`` media queries.

10.1 Breakpoint Breakdown

| Viewport | Device Class                    | Usability State | Observable Breakages & Issues                                                                                                                            |
|----------|---------------------------------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 360px    | Small Android                   | Degraded        | Recent Sessions table blows out horizontally. Lifecycle stepper labels collide. Metric cards wrap into single tall stack. Notification badges clip edge. |
| 390px    | iPhone 14 / Modern Mobile       | Partial         | Sidebar collapses to drawer. Topbar title truncates. Table overflows horizontally. Modals take 98% screen width with awkward inner scroll.               |
| 768px    | Tablet Portrait (iPad)          | Good            | Sidebar visible or collapsible. Dual reporting cards wrap vertically. Verification player fits comfortably. Stepper text readable.                       |
| 1024px   | Tablet Landscape / Small Laptop | Excellent       | Full desktop sidebar layout. Dual reporting columns display side-by-side. Source reference drawer slides out with minor editor narrowing.                |
| 1440px+  | Standard Desktop Monitor        | Optimal         | Primary design target. Spacious margins, perfect side-by-side diff panes, uninterrupted audio scrubbing.                                                 |

10.2 Specific Observable Layout Breakages

- **Recent Sessions Table Horizontal Overflow on Mobile (360px–390px):**

- The `.sessions-table` on `DashboardView` and `SessionHistoryList` contains 8 columns: Date, Title, Programme, Duration, Words, Flags, Status, and Actions.
- On screens narrower than 768px, there is no responsive card transformation or horizontal scroll container wrapper, causing the table to force the parent document width out to ~850px, introducing horizontal viewport wobbling.
- **Lifecycle Stepper Step Labels Clipping:**
- The 8-stage stepper (`LifecycleStepper.jsx`) uses horizontal flex with connecting lines. Below 992px, the text labels below each node ("Master Audio", "Raw STT", "Verification", "Verified Transcript", "Reporting", "Editing", "Proofreading", "Final Report") wrap awkwardly, overlap neighboring nodes, or clip off the right viewport margin.
- **Dual Reporting Comparative Columns Stacked on Tablet:**
- On `ReportingView.jsx`, Reporter A and Reporter B are rendered in a 2-column CSS grid. On screens below 1100px, the grid collapses to 1 column. This breaks the primary UX goal of the screen: comparing the two summaries side-by-side to catch theological omissions.
- **Source Reference Drawer Obscuring Editorial Textarea:**
- On viewports between 768px and 1024px, opening the `SourceReferenceDrawer` (width: 440px) leaves less than 350px of visible space for the main editing textarea, making writing uncomfortable without closing the drawer.

## 11. VISUAL EVIDENCE CATALOG

A dedicated visual asset folder has been established at ``ux-review-screenshots/``. This folder contains 17 high-resolution PNG images comprising live browser renders and the official target mockups from the design phase.

| #  | File Name                                 | Type                 | Dimensions | What It Demonstrates                                                                                      |
|----|-------------------------------------------|----------------------|------------|-----------------------------------------------------------------------------------------------------------|
| 01 | <code>`01-dashboard-desktop.png`</code>   | Live Chrome Render   | 1440 × 900 | Running desktop Dashboard: Sidebar, KPI cards, audio dropzone, recent sessions table, system status pill. |
| 02 | <code>`02-dashboard-mobile.png`</code>    | Live Chrome Render   | 390 × 844  | Running mobile Dashboard: Collapsed drawer menu, stacked metrics, table horizontal overflow behavior.     |
| 03 | <code>`home-dashboard.png`</code>         | Design Target Mockup | 1440 × 900 | Official Stitch design target for Dashboard: Spacious quick actions and recent sessions directory.        |
| 04 | <code>`new-live-session.png`</code>       | Design Target Mockup | 1440 × 900 | Setup form for live microphone capture: Programme/session pickers, VU meter, provider selector.           |
| 05 | <code>`live-session-recording.png`</code> | Design Target Mockup | 1440 × 900 | Full-screen active recording monitor: Big timer, live audio level bar, streaming real-time transcript.    |
| 06 | <code>`session-completion.png`</code>     | Design Target Mockup | 1440 × 900 | Post-recording modal summary: Duration, words captured, flag count, and next-step action cards.           |
| 07 | <code>`sessions-history.png`</code>       | Design Target Mockup | 1440 × 900 | Complete church sermon directory: Filters, status tabs, pagination, and action menus.                     |
| 08 | <code>`session-workspace.png`</code>      | Design Target Mockup | 1440 × 900 | Central session hub: 8-stage stepper, Action Required banner, 6 artifact tiles, and future roadmap.       |

| #  | File Name                    | Type                 | Dimensions | What It Demonstrates                                                                                         |
|----|------------------------------|----------------------|------------|--------------------------------------------------------------------------------------------------------------|
| 09 | `raw-transcript.png`         | Design Target Mockup | 1440 × 900 | Raw STT viewer: Synchronized audio scrubber player with interactive timestamped word segments.               |
| 10 | `verification-workspace.png` | Design Target Mockup | 1440 × 900 | Human verification workbench: 5-second audio snippet loop player, progress bar, flag resolution rows.        |
| 11 | `verified-transcript.png`    | Design Target Mockup | 1440 × 900 | Approved transcript viewer: Clean text preview, verifier audit log, and proceed-to-reporting action.         |
| 12 | `reporting-workspace.png`    | Design Target Mockup | 1440 × 900 | Stage 4 Dual Reporting: Side-by-side comparative cards for Reporter A (thematic) and Reporter B (narrative). |
| 13 | `editing-workspace.png`      | Design Target Mockup | 1440 × 900 | Stage 5 Editorial Synthesis: Editorial workbench with slide-out Source Reference Flyout Drawer.              |
| 14 | `proofreading-workspace.png` | Design Target Mockup | 1440 × 900 | Stage 6 Proofreading Setup: Rule indicators (Scripture citations, divine pronouns) and trigger button.       |
| 15 | `proofreading-review.png`    | Design Target Mockup | 1440 × 900 | Stage 6 Inspection Review: Dual-pane layout showing marked-up text and categorized change list.              |
| 16 | `proofreading-finalized.png` | Design Target Mockup | 1440 × 900 | Approved proofread deliverable: Clean text preview ready for final publication.                              |
| 17 | `final-report.png`           | Design Target Mockup | 1440 × 900 | Stage 7 Final Deliverable: Formatted bulletin preview and Word `.docx` download action.                      |

## 12. CURRENT UX PROBLEMS OBSERVED

To assist ChatGPT in planning the final review and polish, this section categorizes observed UX issues into **clearly observable defects** (unambiguous bugs and anti-patterns) and **context-dependent concerns** (architectural and workflow trade-offs).

### 12.1 Clearly Observable Issues (Bugs, Breakages & Anti-Patterns)

- **Recent Sessions Table Horizontal Overflow on Mobile:**
  - On screens under 768px (`02-dashboard-mobile.png`), `sessions-table` blows out the parent container horizontally. There is no card transformation, sticky column, or smooth horizontal swipe container, creating accidental page zooming and horizontal scrolling.
- **Lifecycle Stepper Step Labels Overlap Below 1024px:**
  - In `LifecycleStepper.jsx`, the 8 pipeline stages are laid out in a single horizontal flex line. On tablet portrait and smaller viewports, text captions collide or run off the right edge.
- **Reliance on System Unicode Emojis Instead of an Icon System:**

- Navigation links, buttons, and badges rely on emojis (👉, 👈, 👉👈, 👈👈, 👉👉, 👈👉, 👉👉, 👈👈). Emojis render wildly differently across operating systems (e.g. Segoe UI on Windows vs Apple Color Emoji on macOS vs Roboto/Noto on Android), causing visual dissonance and alignment inconsistencies.
- **No Browser History / Client-Side URL Routing:**
  - The entire app is driven by React `useState` (`currentView` and `activeView`). Pressing F5 (refresh) immediately resets the operator to the Dashboard, abandoning active sub-views, scroll positions, or uncommitted form drafts. Pressing the browser back button (`Alt + ⬅️`) exits the web application entirely.
- **No Keyboard Shortcuts in Verification Workspace:**
  - Resolving flags in `VerificationWorkflow.jsx` requires dozens of mouse clicks to play/pause audio, scrub timestamps, focus correction inputs, and submit. The lack of standard hotkeys (e.g. `Space` for play/pause, `Enter` to confirm, `Tab` to next item, `Esc` to discard) creates severe fatigue during long sermons with 50+ flags.
- **Plain Textarea Lacks Rich Text / Formatting Controls in Editorial Stage:**
  - In `EditingView.jsx`, the chief editor writes into a plain ``<textarea>`. There is no visual toolbar for bold, italic, bullet lists, or scripture pull-quotes, forcing editors to write manual markdown or copy-paste into external Word processors.
- **Standards Modals Code Duplication:**
  - `ReportingStandardsModal.jsx`, `EditorStandardsModal.jsx`, and `ProofreadingStandardsModal.jsx` contain ~80% duplicated modal boilerplate, tab switching, and version rollback code, increasing maintenance overhead and inconsistent styling.
- **Lack of Confirmation Dialog on Certain Destructive Actions:**
  - In `VerificationWorkflow.jsx`, clicking `Discard / False Positive` immediately removes the flag without confirmation. While undo is technically possible via database inspection, there is no in-app undo toast.
- **Floating Controller Overlaps Interactive Elements on Low-Height Screens:**
  - When recording in the background, `FloatingRecordingController.jsx` is pinned at `bottom: 1.5rem; right: 1.5rem; z-index: 9999;`. On laptop screens with 768px height or mobile devices, it frequently obscures pagination buttons and form save controls.

## 12.2 Context-Dependent Concerns (Workflow & Design Trade-offs)

- **Dual Reporting Layout vs Single Synthesized Summary:**
  - The side-by-side presentation of Reporter A (outline) and Reporter B (narrative) is conceptually brilliant for theological thoroughness, but requires significant horizontal screen width. On screens between 768px and 1200px, stacking them vertically doubles scroll depth and makes direct comparison tedious.
- **Automatic Flag Detection Thresholds:**
  - If confidence threshold is set too conservatively (e.g. < 80%), routine speech disfluencies generate 80+ flags per sermon, overwhelming church operators. If set too strictly, misheard biblical proper names pass unnoticed into downstream reports.
- **Markdown Textarea vs Full WYSIWYG Editor:**
  - Keeping the editor as plain markdown preserves clean semantic data for the backend `.docx` compiler without HTML tag pollution. However, volunteer church secretaries unfamiliar with `#` and `\*\*` markdown syntax may find this intimidating compared to a Google Docs-like toolbar.
- **Direct Prompt Engineering Exposed in Settings:**
  - Allowing operators to edit raw LLM prompts in `SettingsView.jsx` provides unprecedented customization, but risks prompt injection or accidental formatting degradation if an untrained operator deletes prompt guardrails. A guided form with structured rule toggles might be safer than open textareas.
- **Single-Operator Focus vs Distributed Team Workflow:**
  - Currently, a single operator must perform audio technician duties, verification, reporting, and proofreading sequentially. While ideal for a small church team, larger media units with distinct audio teams and editorial committees cannot work concurrently on different sessions without account-based collaboration.

## 13. ACCESSIBILITY REVIEW

This review benchmarks the current implementation against WCAG 2.1 Level AA accessibility standards.

### 13.1 Color Contrast

- **Passing (AAA):**
  - Primary text (`#0f2947`) on pure white background (`#ffffff`): **14.2:1 contrast ratio**.
  - Body text (`#334155`) on white background (`#ffffff`): **9.6:1 contrast ratio**.
  - Primary button background (`#163e73`) with white text (`#ffffff`): **9.8:1 contrast ratio**.
- **Failing / Marginal (Below AA 4.5:1):**
  - Table header labels (`#64748b`) on light grey background (`#f8f9fc`): **4.1:1 contrast ratio** (fails for small text under 14pt).
  - Inactive lifecycle stepper text (`#94a3b8`) on white background: **2.6:1 contrast ratio** (severely fails).
  - Amber badge text (`#d97706`) on light yellow background (`#fef3c7`): **3.2:1 contrast ratio** (borderline, difficult for visually impaired users).

### 13.2 Interactive Touch Targets & Focus States

- **Touch Target Size:**
  - Standard buttons (`.btn--primary`, `.btn--secondary`) meet the 44 × 44px minimum target size.
  - Inline table action buttons (`□□ Rename`, `□ Archive`, `□ Play`) measure approximately 28 × 28px or 32 × 32px, making them difficult to tap accurately on touchscreens.
- **Keyboard Focus Rings:**
  - Standard form `<input>` and `<select>` elements feature clean `:focus` outlines with 3px ice blue rings.
  - Interactive clickable cards (`.stat-card-stitch`, `.artifact-tile`, quick action cards) lack `:focus-visible` styling, making them invisible to keyboard-only tab navigation.

### 13.3 Screen Reader & ARIA Attributes

- **Missing ARIA Roles:**
  - `LifecycleStepper.jsx` is marked up as generic `<div>` elements without `role="progressbar"`, `aria-valuenow`, or `aria-current="step"`.
  - `AudioLevelMeter.jsx` lacks `role="meter"` and `aria-valuenow`.
  - Emoji-only buttons (such as `✖` close buttons and `⏏` maximize buttons) lack `aria-label` attributes, reading as "Multiplication sign" or generic unicode characters to screen readers.
  - Dynamic live transcription stream in `LiveRecordingView.jsx` lacks `aria-live="polite"`, preventing assistive technologies from announcing incoming sermon sentences.

## 14. CONTENT & TERMINOLOGY

This section reviews the linguistic consistency and theological nomenclature across the user interface.

### 14.1 Church & Domain-Specific Nomenclature

The application correctly embraces Deeper Life Bible Church (DLBC) operational phrasing:

- "Programme": Top-level church gatherings (e.g. Sunday Worship Service, Monday Bible Study, Thursday Revival Hour, December National Retreat).

- "Section / Session": Individual service modules within a programme (e.g. Morning Message, Faith Clinic, Youth Choir Session).
- "Minister / Speaker": The preaching pastor (specifically configured for Pastor W.F. Kumuyi).
- "Message Title": The specific sermon theme or sermon topic.
- "Verification": The human-in-the-loop acoustic inspection of church jargon, biblical proper nouns, and Scripture references.
- "Divine Pronouns": Capitalization rules governing reference to Deity ("He", "Him", "His", "Thy", "Thine").

## 14.2 Inconsistent Text Casing & Formatting

- **Button Casing:**
  - Most buttons use Title Case: `Start Live Session & Recording`, `Generate Dual Reports`, `Finalise & Approve Transcript`.
  - Some buttons use Sentence case: `Save draft`, `Cancel`, `Open active recorder`.
  - Certain badges use ALL CAPS: `RECORDING`, `COMPLETED`, `INTERRUPTED`, `NEEDS VERIFICATION`.
  - Recommendation for Polish: Enforce Title Case across all call-to-action buttons and Title Case across badges (`Needs Verification`, `Completed`).
- **Ambiguous Action Terminology:**
  - In `SessionCompletionView.jsx`, the button "Skip to Reporting" actually calls `confirmRawAsVerified()` in the background. A clearer label would be "Approve Without Verification" or "Proceed Directly to Reporting".
  - In `EditingView.jsx`, the button "Approve Editorial Draft" transitions the session to Proofreading. A clearer label would be "Approve & Send to Proofreading".

# 15. IMPLEMENTATION CONSTRAINTS FOR POLISHING

When ChatGPT or the engineering team plans UI/UX modifications, the following hard architectural constraints **must be respected**:

- **React 19 & Vite Execution Model:**
  - The frontend is built on React 19 (`19.0.0`) and Vite. Any newly introduced third-party UI libraries must be strictly compatible with React 19 peer dependencies.
- **Zero Client-Side Routing Overhead:**
  - View navigation is managed by React state in `App.jsx` and `SessionDetailView.jsx`. If implementing a router (such as `wouter` or `react-router`), it must be done cautiously to avoid breaking the background live audio capture session.
- **Monolithic `App.css` Architecture:**
  - Styling is centralized in a single 10,429-line CSS file. Do not attempt a blind rewrite into Tailwind CSS or styled-components during a UI polish; instead, leverage and refine the existing CSS custom properties (`--color-primary`, `--radius-md`, etc.) and modularize styles incrementally.
- **Persistent Web Audio MediaStream Constraints:**
  - Live microphone capture relies on an in-memory `AudioContext` and `MediaRecorder` instance managed by `useAudioCapture.js`. Any UI layout changes to `LiveRecordingView` or `FloatingRecordingController` must preserve the hook's continuous lifecycle. Unmounting the hook or triggering unnecessary re-renders will sever the audio recording stream.
- **Strict Pydantic API Schemas on FastAPI Backend:**
  - The backend enforces strict Pydantic v2 validation contracts across all endpoints. Renaming frontend form fields (e.g. changing `session\_id`, `reporter\_role`, `standard\_version`, `report\_text`) will trigger HTTP 422 Unprocessable Entity errors unless corresponding backend schemas are updated simultaneously.
- **Lossless Data Preservation Mandate:**
  - In accordance with project requirements, raw audio files (`.wav`), raw STT segments, and unedited speech transcriptions must never be overwritten, discarded, or mutated. All verification, editing, and proofreading operations must persist as

non-destructive revisions in their respective database tables.

# 16. SOURCE FILE MAP

This master index maps every screen and UI component to its exact location in the codebase for rapid engineering handoff.

| Category      | Component / View Name            | Exact File Path                                                     |
|---------------|----------------------------------|---------------------------------------------------------------------|
| Shell & Core  | Root Container & Navigation      | `frontend/src/App.jsx`                                              |
| Shell & Core  | Master App Shell & Header        | `frontend/src/components/common/AppShell.jsx`                       |
| Shell & Core  | 8-Stage Lifecycle Stepper        | `frontend/src/components/common/LifecycleStepper.jsx`               |
| Shell & Core  | Global Error Banner              | `frontend/src/components/ErrorBanner.jsx`                           |
| Shell & Core  | Master CSS Stylesheet            | `frontend/src/App.css`                                              |
| Dashboard     | Home Dashboard View              | `frontend/src/components/dashboard/DashboardView.jsx`               |
| Live Audio    | New Live Session Setup           | `frontend/src/components/sessions/NewLiveSessionView.jsx`           |
| Live Audio    | Live Recording Monitor           | `frontend/src/components/recording/LiveRecordingView.jsx`           |
| Live Audio    | Floating Recording Widget        | `frontend/src/components/recording/FloatingRecordingController.jsx` |
| Live Audio    | Audio Level VU Meter             | `frontend/src/components/AudioLevelMeter.jsx`                       |
| Live Audio    | Microphone Device Picker         | `frontend/src/components/AudioSourceSelector.jsx`                   |
| Live Audio    | Completed Recording Audio Player | `frontend/src/components/CompletedRecordingPlayer.jsx`              |
| Sessions      | Session Completion Modal         | `frontend/src/components/sessions/SessionCompletionView.jsx`        |
| Sessions      | Sessions History Directory       | `frontend/src/components/sessions/SessionHistoryList.jsx`           |
| Sessions      | Central Session Workspace Hub    | `frontend/src/components/sessions/SessionDetailView.jsx`            |
| Transcription | Raw Transcript Viewer            | `frontend/src/components/transcription/RawTranscriptViewer.jsx`     |
| Transcription | Recorded File Uploader           | `frontend/src/components/transcription/RecordedFileUploader.jsx`    |
| Transcription | Upload & Transcribe Progress     | `frontend/src/components/transcription/TranscriptionProgress.jsx`   |
| Transcription | Transcripts History Sidebar      | `frontend/src/components/transcription/TranscriptsHistoryList.jsx`  |
| Verification  | Verification Workspace Hub       | `frontend/src/components/verification/VerificationWorkflow.jsx`     |
| Verification  | Flagged Item Row Component       | `frontend/src/components/verification/VerificationItemRow.jsx`      |
| Reporting     | Dual Reporting Workspace         | `frontend/src/components/reporting/ReportingView.jsx`               |
| Reporting     | Individual Reporter Card         | `frontend/src/components/reporting/ReporterCard.jsx`                |
| Reporting     | Reporting Standards Modal        | `frontend/src/components/reporting/ReportingStandardsModal.jsx`     |

| Category     | Component / View Name             | Exact File Path                                                       |
|--------------|-----------------------------------|-----------------------------------------------------------------------|
| Editing      | Editorial Synthesis Workbench     | `frontend/src/components/editing/EditingView.jsx`                     |
| Editing      | Source Reference Flyout Drawer    | `frontend/src/components/editing/SourceReferenceDrawer.jsx`           |
| Editing      | Editor Standards Modal            | `frontend/src/components/editing/EditorStandardsModal.jsx`            |
| Proofreading | Proofreading Workspace & Diff     | `frontend/src/components/proofreading/ProofreadingView.jsx`           |
| Proofreading | Proofreading Changes List         | `frontend/src/components/proofreading/ProofreadingChangesList.jsx`    |
| Proofreading | Proofreading Standards Modal      | `frontend/src/components/proofreading/ProofreadingStandardsModal.jsx` |
| Final Report | Final Report Preview & Export     | `frontend/src/components/final_report/FinalReportView.jsx`            |
| YouTube      | YouTube Ingestion & Tab Capture   | `frontend/src/components/youtube/YouTubeSessionView.jsx`              |
| Settings     | Settings & Standards Hub          | `frontend/src/components/settings/SettingsView.jsx`                   |
| Settings     | Church Programmes Management      | `frontend/src/components/settings/ProgrammesSettingsSection.jsx`      |
| Hooks        | Audio Capture & Web Audio Hook    | `frontend/src/hooks/useAudioCapture.js`                               |
| Hooks        | Recorded Transcription Hook       | `frontend/src/hooks/useRecordedTranscription.js`                      |
| Hooks        | Church Sessions CRUD Hook         | `frontend/src/hooks/useSessions.js`                                   |
| Backend      | Database DDL & Schema Models      | `backend/app/database/models.py`                                      |
| Backend      | Sessions Database Repository      | `backend/app/database/session_repo.py`                                |
| Backend      | Reporting Database Repository     | `backend/app/database/reporting_repo.py`                              |
| Backend      | Editing Database Repository       | `backend/app/database/editing_repo.py`                                |
| Backend      | Proofreading Database Repository  | `backend/app/database/proofreading_repo.py`                           |
| Backend      | Final Report Word .docx Generator | `backend/app/database/final_report_repo.py`                           |
| Backend      | Church Programmes Repository      | `backend/app/database/programmes_repo.py`                             |
| Backend      | Interruption & Crash Recovery     | `backend/app/database/interruption_recovery.py`                       |
| Backend      | FastAPI Entrypoint & API Routers  | `backend/app/main.py`                                                 |

## 17. SECURITY REDACTION SUMMARY

In strict adherence to security non-negotiables, all sensitive credentials, secret tokens, and connection strings have been verified as redacted or excluded from this handoff document:

- **Azure Speech Services:** Secret subscription keys (`AZURE\_SPEECH\_KEY`) are not disclosed. Only the Azure datacenter region (`southafricanorth`) and sample rate are noted.
- **Google Gemini API:** Cloud AI API keys (`GEMINI\_API\_KEY`) remain in local `.env` files and are never exported to markdown or frontend builds.
- **SQLite Database:** Database access path is internal (`storage/app.db`) with zero remote network port exposure.
- **Personally Identifiable Information (PII):** Mock names and sample citations use publicly known pastoral references (Pastor W.F. Kumuyi) and canonical King James Scripture texts.

End of DLBC Information Unit UI/UX Comprehensive Handoff Document.

# 18. Takeover Audit, 30 September 2026

## Current audit and recommendations

### Baseline and evidence boundary

- Repository: ``https://github.com/isabeldboy1126-design/DLBC-Information-Unit.git``.
- Working branch: ``codex/product-ux-audit``, created from the remote default ``master`` at ``b809f9a``.
- At clone time, ``origin/master``, ``origin/main``, and ``origin/experiment/neutral-dark-theme`` point to the same commit. Their names do not represent different current implementations.
- Frontend dependencies installed from the existing lockfile using ``npm ci --ignore-scripts --no-audit --no-fund``. No dependencies were added or upgraded.
- ``npm run build``: passed. ``npm run lint``: exit 0 with existing warnings, including hook dependency and unused-variable warnings. This is not a warning-free result.
- Local frontend: ``http://127.0.0.1:5173/``, with backend and AI providers kept offline.
- Browser inspection: dashboard, session history, settings, and new-live-session setup at 1440 × 900; dashboard and new-live setup at 390 × 844; mobile navigation focus; browser Back and refresh on the setup route.
- No source recordings, uploads, microphone grants, AI processing calls, backend tests, or live data were used. Successful populated-session workflows, save persistence, actual transcription, export, dark theme, contrast, and reduced motion remain unverified.
- This pass changes this handoff only. No application code, product scope, milestone acceptance, commits, pushes, or deployments are changed.

### Product reading and proposed direction

The documented product is a church-internal, desktop-first reporting tool. Its core value is reducing manual reporting effort while keeping sermon sources traceable and human review explicit. The project documents identify the owner as the confirmed initial operator; broader stakeholder adoption remains unconfirmed.

The design opportunity is a dependable operator workspace: make source availability, review obligations, saved work, and the next action clear for each session. Retain the recognisable shell, session-based organisation, audio permission explanation, source reference tools, and existing report-stage components. Keep the existing blue identity as a starting constraint; palette replacement and a new architecture are not needed for the first slice.

This is a proposed direction grounded in the specification and inspected implementation. It is not evidence from operator interviews or real-service observation.

### Findings and priorities

**P1: Reconcile automatic generation with human approval before redesigning completion states.** The current processing engine describes reporter extraction, editorial selection, writing, and proofreading inside a single processing path. It then calls ``finalize_report`` with ``proofread_report_revision_id=None``; the repository function sets ``final_report_status='complete``. This differs from the separate editor/proofreader stages and human review in ``PROJECT_SCOPE.md`` and ``AGENTS.md``. The offline settings screen also shows Auto-Process After Verification enabled, and both frontend initial state and backend settings seeding default it to true. That toggle is distinct from the older auto-continue-to-proofreading setting, so their semantics must be reconciled explicitly. Evidence: **engine, final-report repository, settings view, settings seeding**. Source finding; the processing path was not run. Proposed contract: distinguish AI-generated output awaiting review from a human-approved final report. Resolve whether the consolidated pipeline is an intentional product change before modifying stage logic or approval behaviour.

**P1: Unavailable data is presented as an empty collection.** With the backend offline, the dashboard displays ``Notice: Failed to fetch`` together with ``No recorded sessions found``; history displays ``No Sessions Found`` and says no sessions have been recorded. The request failure cannot establish that the collection is empty. ``useSessions`` begins with an empty array, and the dashboard is not given a collection load/error state. Evidence: **session hook, app wiring**, and the local browser captures. Give the collection its own loading, loaded-empty, loaded-with-data, and unavailable states, with retry and

truthful copy. When refreshing known data fails, retain it and label it as previously loaded.

**P1: The dashboard asserts hardware readiness without checking it.** `Mic / USB Ready` is unconditional dashboard content. It appears even though the new-live screen says microphone permission is required and no permission was granted during this audit. Evidence: **dashboard**, browser comparison of dashboard and setup. Replace the claim with a neutral setup action until permission and device state are known. Distinguish audio input availability from backend connection and transcription-provider readiness.

**P1: Automation settings can appear saved when they are not.** The offline settings screen renders an enabled, checked automation toggle despite not loading its persisted value. Its handler immediately changes local state, does not inspect `response.ok`, and only logs a fetch exception without reverting the value or showing a save error. Evidence: **SettingsView**, initial offline browser state. The handler finding is from source; no setting was submitted. Represent unknown persisted values explicitly, prevent edits until loaded, and show saving, saved, and failed states with rollback/retry.

**P1: Mobile navigation leaves keyboard focus behind its overlay.** At 390 × 844, opening the drawer leaves focus on Open navigation. Pressing Tab focuses the Back button behind the drawer. The page content remains in the accessibility tree; there is no drawer focus containment or background inertness in the inspected implementation. Evidence: browser reproduction and **AppShell**. Move focus into the drawer on open, keep keyboard interaction within it while it overlays the page, make covered content inactive, and return focus to the opener on close. Escape dismissal already exists and should be retained.

**P2: The brand home control lacks keyboard activation.** On the settings screen, focusing the DLBC brand control and pressing Enter leaves the route unchanged. The element is a `div` with `role="button"`, `tabIndex`, and a click handler, but no key handler. Evidence: browser reproduction and **AppShell**. Use a native actionable element with the same appearance and a clear accessible name.

**P2: Desktop and mobile report completion rules differ.** Desktop recent-session rows treat either `final\_report\_status='complete'` or `report\_processing\_status='completed'` as final completion. Mobile rows only check `final\_report\_status`. This can produce different labels/actions for the same data when those fields differ. Evidence: **DashboardView**, source only; no populated session was available to reproduce the disagreement. Centralise presentation-state derivation after the approval contract is settled, and verify it with representative session fixtures.

**P2: Dashboard action hierarchy does not match the documented V1 boundary.** The existing-file upload path is a small text action beneath two large live/YouTube cards. The scope describes live capture and existing recordings as V1 paths, while direct YouTube downloading is deferred. Source confirms a YouTube implementation exists; that alone does not establish whether a scope change was approved. Evidence: **scope, dashboard, YouTube view**. Give Record live and Upload recording comparable discoverability. Confirm the intended status of YouTube before moving, expanding, or removing it.

**P2: The notification affordance implies functionality that is absent.** The bell has a persistent indicator dot and an accessible Notifications label, but no click handler or notification content in `AppShell`. Evidence: source inspection. Connect it to a real actionable notification state or remove the unsupported indicator/control in an approved interface slice.

## What to keep and what earlier claims no longer establish

- Browser history routing is now implemented. Navigating from setup to dashboard, using browser Back, and refreshing returned to/preserved `#new\_live` in this pass. The earlier blanket claim that the app has no routing is superseded. This check does not prove recovery of unsaved drafts or active recordings.
- Current dashboard has a five-column desktop table and a separate mobile session presentation. The inspected empty dashboard and setup measured document width 390 at viewport width 390. The earlier claim of universal dashboard table overflow should not be carried forward. Populated history, long titles, and the session stepper still need separate checks.
- The shell uses several SVG icons and a shared confirmation component with focus handling. Earlier statements that all navigation relies on emojis or that no focus handling exists are too broad for the current source. Some secondary screens still use emojis, and the mobile drawer has a specific focus defect.
- The phase plan still labels Complete Application UX and UI Integration and Polish as Not Started, while substantial implementation exists. Treat those labels as unverified delivery records, not a reason to rebuild the shell or declare the phases accepted.
- Original-audio preservation, immutable raw transcript, and traceability remain product requirements. Their existence in documents or source is not proof of end-to-end reliability on this machine.

## Recommended improvement slices

- **Truthful dashboard and session loading.** Show connection/load state, make Record live and Upload recording clear, separate unavailable from empty data, and provide retry. Preserve current navigation and data contracts. Acceptance: an offline backend never claims no sessions exist or that audio hardware is ready; a successful empty response gets first-session guidance; existing sessions remain usable after a failed refresh where feasible.
- **Reliable setup and accessible shell.** Make programme prerequisites and permission/device checks explicit, fix keyboard activation and drawer focus, and remove unsupported notification signals. Acceptance: a keyboard user can enter/exit navigation and complete available setup steps; covered page controls cannot receive focus through the open drawer; backend failure is not described as missing programme configuration.
- **Review workspace and final approval.** After the pipeline decision, align attention queues, progress indicators, editable drafts, source/audio references, and final-review language around one agreed contract. Acceptance: users can distinguish processing success from human approval, identify what requires review, and recover work. This slice requires backend-backed representative sessions and project-native regression evidence before any completion claim.

Do the first bounded slice before a broad visual rewrite. Reuse existing tokens and components, then improve typography, spacing, density, and responsive behaviour where the task needs it. Do not add multi-user roles, analytics, new AI capabilities, cloud infrastructure, or integrations merely to make the transformation feel larger.

## Local evidence and next verification

Screenshots are saved in the existing ignored ``ux-review-screenshots/`` directory. They are local evidence and will not accompany a normal Git commit unless separately included intentionally:

- ``dashboard-desktop.png``, ``dashboard-mobile.png``
- ``sessions-offline-desktop.png``, ``settings-offline-desktop.png``
- ``new-live-offline-desktop.png``, ``new-live-mobile.png``
- ``mobile-drawer-focus.png``

Next engineering evidence should use an isolated local backend/test environment with representative sessions in each stage, without real church audio or provider spending. Verify collection load states, setting persistence failures, review/approval transitions, populated responsive layouts, export, and source retention. Backend dependency installation and service execution are outside this pass's frontend-only setup approval.